

# Causal Index-Set Calculus for Boolean Networks and Deterministic Reconstruction

Alberto Hernández-Espinosa

## Abstract

We introduce a causality-first index-set calculus for Boolean networks in which the basic object is not a sampled trajectory or a statistical summary, but the exact set of rules that reconstructs network behaviour from local gate semantics over ordered exhaustive repertoires. The calculus yields closed-form one-sets for monotone, parity, implication, threshold, and canalising gate families; relates MSB-first and LSB-first enumerations by an explicit bit-reversal involution; and reconstructs mixed-gate network outputs exactly by column-wise composition of local index sets. Overlap among queried gate supports is exploited analytically to compress the effective evaluation space from  $2^{d_q}$  to  $2^{\mu_q}$ , where  $\mu_q$  counts repeated coordinates. Deterministic benchmarks confirm exact equality to exhaustive baselines at medium sizes ( $n \leq 13$ ), exact sampled equality at larger sizes ( $n \in \{20, 50\}$ ), and analytic evaluation of the order of a millisecond, independent of  $n$ , at  $n = 200$  under bounded in-degree. The method provides a mathematically transparent alternative to brute-force repertoire enumeration and to purely probabilistic surrogates when the generating mechanism is known.

## 1 Introduction

Since classical mechanics, physics and the natural sciences have treated causal structure, the identification of generative laws behind observed phenomena, as the proper object of explanation, distinct from mere correlation or curve-fitting. Contemporary machine learning and statistical AI, by contrast, largely approach causal structure indirectly: predictive models are fit under strong distributional or parametric assumptions, and causal claims are recovered only approximately, conditional on those assumptions holding. Judea Pearl’s causal calculus formalised this gap within statistics itself, showing that interventionist causal claims require machinery beyond observational correlation [5].

On the other hand, algorithmic probability, rooted in the foundational work of Solomonoff, Kolmogorov, and Chaitin [6, 3, 2], and developed as algorithmic information dynamics [10, 11], is closely aligned with the Principle of Computational Equivalence (PCE) [8], which holds that short generative rules are, in principle, responsible for the full complexity observed in natural and computational systems. While PCE is a thesis about the world, the algorithmic-causality framework is an inferential and interventionist calculus for recovering such rules from observations [10]. The present work aims at a concrete materialisation of the same underlying idea: it uses Boolean networks as an explicit artefact in which small local rules are shown, deterministically and in closed form, to generate the full complexity of the network’s behaviour.

The central difficulty in Boolean-network analysis is therefore not lack of computational power; it is lack of causal compression. One can always enumerate a repertoire, print a truth table, or run a simulation. What remains elusive is a representation that is at once exact, compositional, and explanatory. For a high-level scientific treatment, this distinction matters: a method that merely reproduces outputs is weaker than a method that identifies the rule structure responsible for them.

This paper advances the following thesis. Ordered exhaustive repertoires contain a non-trivial algebraic structure, and this structure can be exploited to replace brute-force enumeration by

exact causal formulae. Once the ordering convention is fixed, the question “what can this network do?” can be recast as “at which conditions is each local rule active, and how do those conditions compose?” The result is an index-set calculus for Boolean networks.

A terminological clarification is needed at the outset. The word “causal” is used in this paper in the *mechanistic* or *generative* sense: the Boolean network is the causal mechanism, and the index-set formula reconstructs the exact input–output map of that mechanism. This usage follows the tradition of mechanistic explanation in philosophy of science [4, 9] and of generative modelling in algorithmic information theory [10], rather than the interventionist *do*-calculus of Pearl [5]. The present calculus does not perform causal discovery from observational data, nor does it reason about interventions or confounders. It derives the exact consequences of a *known* generative rule. Where Pearl’s framework asks “what would happen if we intervened on variable  $X$ ?”, the present framework asks “at which repertoire positions does the known rule for node  $i$  produce output one, and how do these positions compose across the network?”

The perspective is causality-first in three senses. First, it is rule-first rather than correlation-first: the object of study is the generative logic of the system, not only its observed frequencies, the same distinction that separates physical law from statistical regression. Second, it is deterministic rather than statistical: the method aims at exact reconstruction under known dynamics, not at approximate inference under unknown mechanisms, departing from the assumption-laden regime of modern statistical learning. Third, it is compositional: global behaviour is built from local causal rules rather than from opaque end-to-end approximations.

This perspective connects two traditions. The first is PCE [8], which demonstrates that simple discrete rules generate the full complexity observed in natural and computational systems. The second is the algorithmic-causality programme [10, 11], which treats complex systems as computational objects and provides an intervention-oriented calculus for discovering and steering hidden generating mechanisms from observations, emphasising algorithmic probability, generative mechanisms, and causation over surface statistics.

While the present contribution is inspired by both traditions, its objective is distinct. We assume the local Boolean rules and network structure are given, and we derive exact mathematical expressions for the resulting behaviour over the full ordered repertoire. In that sense, the method is less a causal-discovery calculus than a causal-expression calculus. It remains connected to algorithmic probability and to the idea that simple rules can underwrite rich behaviour, but it is more law-like in the narrow physical sense: the target is a closed-form deterministic description of behaviour, not an approximate recovery of an unknown generator. This is precisely why the work is also close in spirit to Wolfram’s emphasis on exact discrete rules [8], while differing from purely statistical or machine-learning approaches that learn predictive regularities without recovering the underlying analytic form [5].

In what follows, we show that ordered exhaustive repertoires admit an algebraic structure that supports exact, network-aware one-set formulae for all canonical gate families; that ordering changes are controlled by an explicit involution, rendering exactness invariant under representation; and that mixed-gate synchronous networks can be reconstructed exactly by composing local index sets. The resulting complexity improvements arise not by removing the  $2^n$  state space but by replacing brute-force gate evaluation with explicit causal formulae, enabling exact query answering without full matrix materialisation, and reducing the validation burden through principled sampling.

Sections 2 and 3 define the ordered-repertoire formalism and the index objects needed for exact reasoning. Section 4 derives the gate-family one-sets and their expanded forms, and presents worked examples including a detailed AND deconvolution, a composed XOR case, and a 10-node mixed-gate benchmark with overlap analysis and dynamical enrichment. Section 5 proves ordering invariance, and Section 6 characterises the scalability and resource envelope of exact causal evaluation together with a validation summary. The appendices collect exact gate expansions, sensitivity formulae, the validation artefact index, and an extended experimental

programme. All computational experiments are reproducible via the companion code repository (see Appendix C for details).

## 2 Formal Setup

Let  $cm \in \{0, 1\}^{n \times n}$  be the connectivity matrix of a Boolean network, where  $cm_{ij} = 1$  means that node  $j$  feeds node  $i$ . Let

$$dynamic = (d_1, \dots, d_n)$$

be the list of local gate labels, and let

$$N_i = \{j \in \{1, \dots, n\} : cm_{ij} = 1\}$$

be the connected-input set of node  $i$ . Each node has a local map

$$g_i : \{0, 1\}^{|N_i|} \rightarrow \{0, 1\},$$

and the synchronous one-step network update is

$$F(x) = (g_1(x_{N_1}), \dots, g_n(x_{N_n})), \quad x \in \{0, 1\}^n.$$

**Definition 1** (Ordered Exhaustive Repertoire). The ordered exhaustive repertoire of size  $n$  is the set  $\{0, 1\}^n$  together with a declared enumeration of all  $2^n$  binary vectors. We write  $\mathcal{U}_n = \{1, \dots, 2^n\}$  for the corresponding index universe.

Two orderings are operationally relevant throughout the project.

- **LSB-first ordering:** the index  $j$  is associated with  $v(j) = \text{Reverse}(\text{IntegerDigits}(j - 1, 2, n))$ , with weights  $2^{i-1}$ .
- **MSB-first ordering:** the index  $j$  is associated with  $v(j) = \text{IntegerDigits}(j - 1, 2, n)$ , with weights  $2^{n-i}$ .

**Definition 2** (Bit-Reversal Involution). For  $j \in \mathcal{U}_n$ , define

$$\varphi(j) = 1 + \text{FromDigits}\left(\text{Reverse}(\text{IntegerDigits}(j - 1, 2, n)), 2\right).$$

Then  $\varphi : \mathcal{U}_n \rightarrow \mathcal{U}_n$  is an involution.

The involution  $\varphi$  is not a cosmetic convenience. It is the formal device that makes ordering control explicit. Without it, one risks confusing genuine mathematical differences with mere encoding differences.

## 3 Causal Index Sets

The method begins with a structural observation: each coordinate of an ordered exhaustive repertoire forms periodic bands, and Boolean gates can be expressed as exact set operations on those bands.

**Definition 3** (Bit Bands). For a node index  $i \in \{1, \dots, n\}$  and a bit value  $a \in \{0, 1\}$ , define

$$\mathcal{B}_i^a = \{j \in \mathcal{U}_n : v_i(j) = a\}.$$

**Definition 4** (Parity and Weight Classes). For a connected-input set  $I_c \subseteq \{1, \dots, n\}$ , define

$$\mathcal{P}_{I_c}^1 = \left\{ j \in \mathcal{U}_n : \sum_{i \in I_c} v_i(j) \equiv 1 \pmod{2} \right\},$$

$$\mathcal{P}_{I_c}^0 = \left\{ j \in \mathcal{U}_n : \sum_{i \in I_c} v_i(j) \equiv 0 \pmod{2} \right\},$$

and for  $r \in \{0, \dots, |I_c|\}$ ,

$$\mathcal{H}_{I_c}^r = \left\{ j \in \mathcal{U}_n : \sum_{i \in I_c} v_i(j) = r \right\}.$$

**Definition 5** (Decimal Weights, Anchors, and Sumandos). Let the ordering policy be fixed. Define the bit weight

$$w(i) = \begin{cases} 2^{i-1}, & \text{LSB-first,} \\ 2^{n-i}, & \text{MSB-first.} \end{cases}$$

For  $S \subseteq \{1, \dots, n\}$  and  $\eta \in \{0, 1\}^{|S|}$ , define the offset

$$\Delta_S(\eta) = \sum_{t \in S} \eta_t w(t),$$

and for a connected-input set  $I_c$ , define the AND decimal anchor

$$P(I_c) = \sum_{i \in I_c} w(i).$$

These three families already encode most of the method. Bands capture fixed-bit conditions, parity classes capture XOR-like structure, and Hamming-weight strata capture threshold logic. The following sections show that standard gate families are simple set expressions built from them.

**Definition 6** (Deconvolution Operator). For finite sets  $L, S \subset \mathbb{Z}$ , define

$$\text{Dec}(L, S) = \{\ell + s : \ell \in L, s \in S\}.$$

This operator unfolds a compressed pair  $(L, S)$  into the full set of repertoire indices. When  $L$  is a base set of decimal anchors and  $S$  is the offset family generated by the free coordinates,  $\text{Dec}(L, S)$  recovers the exact one-set without row-wise gate evaluation.

**Remark 1** (Terminology: Deconvolution versus Minkowski Sum). The set-addition form  $\{\ell + s : \ell \in L, s \in S\}$  resembles the Minkowski sum of two point sets in  $\mathbb{Z}$ . The term *deconvolution* is used here advisedly, because the operation reverses the combinatorial convolution that produces a composite repertoire index from independent subsystem contributions: the base set  $L$  encodes the coordinates the schema constrains, the offset set  $S$  encodes those it leaves free, and  $\text{Dec}$  reconstructs the full index set from these two factors. Freedom here is a property of the schema, not of the node: the disconnected coordinates are free in every schema and so lie in  $S$  always, but a connected input unconstrained by a particular schema lies in  $S$  too. Rule 110, whose three inputs are all connected yet which decomposes as  $01*$ ,  $10*$ ,  $*10$ , is the standing witness that  $S$  is not the disconnected set. In signal-processing terms, the composite index is the “convolved” signal and  $\text{Dec}$  recovers the constituent structure. The distinction from a generic Minkowski sum is that the two factors here have a definite causal interpretation (constrained versus free coordinates) rather than being arbitrary point sets in a metric space.

## 4 Exact Gate Formulae

The following proposition collects the exact one-set formulae for the gate families supported by the calculus.

**Proposition 1** (Gate-Family One-Sets). *Let node  $k$  have connected-input set  $I_c$ . Under a fixed repertoire ordering, the set of indices at which node  $k$  outputs one is given as follows.*

| Gate family                                                                                 | Exact one-set formula                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AND                                                                                         | $\mathcal{J}_k^{\text{AND}} = \bigcap_{i \in I_c} \mathcal{B}_i^1$                                                                                                                           |
| OR                                                                                          | $\mathcal{J}_k^{\text{OR}} = \bigcup_{i \in I_c} \mathcal{B}_i^1$                                                                                                                            |
| NAND                                                                                        | $\mathcal{J}_k^{\text{NAND}} = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{AND}}$                                                                                                           |
| NOR                                                                                         | $\mathcal{J}_k^{\text{NOR}} = \bigcap_{i \in I_c} \mathcal{B}_i^0 = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{OR}}$                                                                       |
| XOR                                                                                         | $\mathcal{J}_k^{\text{XOR}} = \mathcal{P}_{I_c}^1$                                                                                                                                           |
| XNOR                                                                                        | $\mathcal{J}_k^{\text{XNOR}} = \mathcal{P}_{I_c}^0$                                                                                                                                          |
| NOT on input $r$                                                                            | $\mathcal{J}_k^{\text{NOT}} = \mathcal{B}_r^0$                                                                                                                                               |
| IMPLIES on pair $(a, b)$                                                                    | $\mathcal{J}_k^{\text{IMP}} = \mathcal{B}_a^0 \cup (\mathcal{B}_a^1 \cap \mathcal{B}_b^1)$ , equivalently $\mathcal{U}_n \setminus (\mathcal{B}_a^1 \cap \mathcal{B}_b^0)$                   |
| NIMPLIES on pair $(a, b)$                                                                   | $\mathcal{J}_k^{\text{NIMP}} = \mathcal{B}_a^1 \cap \mathcal{B}_b^0$                                                                                                                         |
| KOFN threshold                                                                              | $\mathcal{J}_k^{\text{KOFN}} = \bigcup_{r=k}^{ I_c } \mathcal{H}_{I_c}^r$                                                                                                                    |
| Strict-EXACT- $k$                                                                           | $\mathcal{J}_k^{\text{EXACT-}k} = \mathcal{H}_{I_c}^k$                                                                                                                                       |
| Canalising with trigger set $C$ , trigger output $o$ , and fallback one-set $\mathcal{J}_h$ | $\mathcal{J}_k^{\text{CAN}} = C \cup ((\mathcal{U}_n \setminus C) \cap \mathcal{J}_h)$ if $o = 1$ ; $\mathcal{J}_k^{\text{CAN}} = (\mathcal{U}_n \setminus C) \cap \mathcal{J}_h$ if $o = 0$ |

*Proof.* We verify each gate family by establishing a bijection between its standard Boolean semantics and the stated set expression.

**AND/OR.** By definition,  $g^{\text{AND}}(x_{N_k}) = 1$  if and only if  $x_i = 1$  for every  $i \in I_c$ . The set of indices satisfying this condition is exactly  $\bigcap_{i \in I_c} \mathcal{B}_i^1$ . The OR case follows by duality:  $g^{\text{OR}} = 1$  iff at least one connected input equals one, which is  $\bigcup_{i \in I_c} \mathcal{B}_i^1$ .

**NAND/NOR.** These are complements:  $\mathcal{J}_k^{\text{NAND}} = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{AND}}$  and  $\mathcal{J}_k^{\text{NOR}} = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{OR}} = \bigcap_{i \in I_c} \mathcal{B}_i^0$ , since requiring all connected inputs to be zero is equivalent to excluding every one-band.

**XOR/XNOR.** The XOR output equals one exactly when the number of active connected inputs is odd:  $\sum_{i \in I_c} v_i(j) \equiv 1 \pmod{2}$ , which is the definition of  $\mathcal{P}_{I_c}^1$ . XNOR is the complementary parity class.

**NOT.** With a single designated input  $r$ , the output is one exactly when  $v_r(j) = 0$ , giving  $\mathcal{B}_r^0$ .

**IMPLIES/NIMPLIES.** For the ordered pair  $(a, b)$ , non-implication  $a \wedge \neg b$  holds exactly on  $\mathcal{B}_a^1 \cap \mathcal{B}_b^0$ . Implication is the complement of this set in  $\mathcal{U}_n$ .

**KOFN/exact- $k$ .** The output equals one when  $\sum_{i \in I_c} v_i(j) \geq k$ , i.e. when the Hamming weight on the connected inputs lies in layers  $\{k, k+1, \dots, |I_c|\}$ . This is  $\bigcup_{r=k}^{|I_c|} \mathcal{H}_{I_c}^r$ . The exact- $k$  variant restricts to the single layer  $\mathcal{H}_{I_c}^k$ .

**Canalising.** Let the trigger set be  $C$ . When the canalising input takes its trigger value, the output is forced to  $o$ . On the complement  $\mathcal{U}_n \setminus C$ , the output follows the fallback rule with one-set  $\mathcal{J}_h$ . If  $o = 1$ , the full one-set is  $C \cup ((\mathcal{U}_n \setminus C) \cap \mathcal{J}_h)$ ; if  $o = 0$ , the trigger band forces zero, so the one-set is  $(\mathcal{U}_n \setminus C) \cap \mathcal{J}_h$ .  $\square$

**Remark 2.** The decisive point is that these are *network-aware* formulae. The local rule is not written abstractly over unnamed inputs; it is anchored to the actual connected-input set  $I_c$  induced by the network topology.

## 4.1 Expanded Gate Expressions

This section introduces the explicit derived expressions for each of the supported gates for the calculation.

**AND.** Let  $I_{nc} = \{1, \dots, n\} \setminus I_c$ . The connected bits are fixed to one, so the decimal contribution of the connected part is the decimal anchor

$$P(I_c) = \sum_{i \in I_c} w(i).$$

The disconnected bits provide the free offset

$$\Delta_{nc}(\eta) = \sum_{t \in I_{nc}} \eta_t w(t), \quad \eta \in \{0, 1\}^{|I_{nc}|}.$$

Hence the one-set can be written as

$$\mathcal{J}_k^{\text{AND}} = \left\{ 1 + P(I_c) + \Delta_{nc}(\eta) \mid \eta \in \{0, 1\}^{|I_{nc}|} \right\},$$

equivalently

$$\mathcal{J}_k^{\text{AND}} = \{j \in \mathcal{U}_n : \forall i \in I_c, v_i(j) = 1\}.$$

**OR.** For each connected input  $i \in I_c$ , define the one-band

$$\mathcal{B}_i^{(1)} = \left\{ 1 + w(i) + \Delta_{\{1, \dots, n\} \setminus \{i\}}(\eta) \mid \eta \in \{0, 1\}^{n-1} \right\}.$$

Then

$$\mathcal{J}_k^{\text{OR}} = \bigcup_{i \in I_c} \mathcal{B}_i^{(1)} = \{j \in \mathcal{U}_n : \exists i \in I_c, v_i(j) = 1\}.$$

**NAND.** The zero-set of NAND is exactly the one-set of AND, so

$$\mathcal{Z}_k^{\text{NAND}} = \mathcal{J}_k^{\text{AND}} = \left\{ 1 + P(I_c) + \Delta_{nc}(\eta) \mid \eta \in \{0, 1\}^{|I_{nc}|} \right\},$$

and the one-set is its complement:

$$\mathcal{J}_k^{\text{NAND}} = \mathcal{U}_n \setminus \mathcal{Z}_k^{\text{NAND}}.$$

**NOR.** For each  $i \in I_c$ , define the zero-band

$$\mathcal{B}_i^{(0)} = \left\{ 1 + \Delta_{\{1, \dots, n\} \setminus \{i\}}(\eta) \mid \eta \in \{0, 1\}^{n-1} \right\},$$

which enforces  $v_i = 0$ . Then

$$\mathcal{J}_k^{\text{NOR}} = \bigcap_{i \in I_c} \mathcal{B}_i^{(0)} = \{j \in \mathcal{U}_n : \forall i \in I_c, v_i(j) = 0\} = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{OR}}.$$

**XOR and XNOR.** These families are controlled by the parity of the connected bits:

$$\mathcal{J}_k^{\text{XOR}} = \left\{ j \in \mathcal{U}_n \mid \sum_{i \in I_c} v_i(j) \equiv 1 \pmod{2} \right\},$$

$$\mathcal{J}_k^{\text{XNOR}} = \left\{ j \in \mathcal{U}_n \mid \sum_{i \in I_c} v_i(j) \equiv 0 \pmod{2} \right\} = \mathcal{U}_n \setminus \mathcal{J}_k^{\text{XOR}}.$$

No decimal-anchor-plus-sumandos decomposition is needed here because the relevant invariant is parity rather than a single threshold band.

**NOT.** For a designated input index  $r$ , the NOT one-set is the zero-band of that input:

$$\mathcal{J}_k^{\text{NOT}} = \mathcal{B}_r^0 = \left\{ 1 + \Delta_{\{1, \dots, n\} \setminus \{r\}}(\eta) \mid \eta \in \{0, 1\}^{n-1} \right\}.$$

**IMPLIES and NIMPLIES.** For an ordered pair  $(a, b)$ , the forbidden asymmetric pattern is  $a = 1, b = 0$ . Its index set is

$$\mathcal{B}_{a=1, b=0} = \left\{ 1 + w(a) + \Delta_{\{1, \dots, n\} \setminus \{a, b\}}(\eta) \mid \eta \in \{0, 1\}^{n-2} \right\}.$$

Therefore

$$\mathcal{J}_k^{\text{NIMP}} = \mathcal{B}_{a=1, b=0}, \quad \mathcal{J}_k^{\text{IMP}} = \mathcal{U}_n \setminus \mathcal{B}_{a=1, b=0}.$$

Equivalently,

$$\mathcal{J}_k^{\text{IMP}} = \{j \in \mathcal{U}_n : \neg v_a(j) \vee v_b(j)\}, \quad \mathcal{J}_k^{\text{NIMP}} = \{j \in \mathcal{U}_n : v_a(j) = 1, v_b(j) = 0\}.$$

**KOFN threshold family.** For threshold  $k$ ,

$$\mathcal{J}_k^{\text{KOFN}} = \bigcup_{r=k}^{|I_c|} \mathcal{H}_{I_c}^r = \left\{ j \in \mathcal{U}_n \mid \sum_{i \in I_c} v_i(j) \geq k \right\}.$$

For strict-exact thresholding,

$$\mathcal{J}_k^{\text{=}} = \mathcal{H}_{I_c}^k = \left\{ j \in \mathcal{U}_n \mid \sum_{i \in I_c} v_i(j) = k \right\}.$$

Thus threshold families are controlled by Hamming-weight layers rather than by parity or a single decimal anchor. The MAJORITY gate is the special case  $k = \lceil |I_c|/2 \rceil$ , so its one-set is the union of weight layers at or above the strict majority threshold.

**Canalising family.** Let the canalising input be  $c$ , the canalising value be  $\nu \in \{0, 1\}$ , the trigger output be  $o \in \{0, 1\}$ , and the fallback one-set be  $\mathcal{J}_h$ . Define the trigger set

$$C_{c, \nu} = \{j \in \mathcal{U}_n : v_c(j) = \nu\}.$$

Then the one-set is

$$\mathcal{J}_k^{\text{CAN}} = \begin{cases} C_{c, \nu} \cup ((\mathcal{U}_n \setminus C_{c, \nu}) \cap \mathcal{J}_h), & o = 1, \\ (\mathcal{U}_n \setminus C_{c, \nu}) \cap \mathcal{J}_h, & o = 0. \end{cases}$$

In the current implementation the fallback rule is OR on the remaining inputs, so  $\mathcal{J}_h$  is typically an OR-type union of one-bands outside the trigger case.

## 4.2 Dynamic Properties and Pattern Taxonomy

The exact one-set calculus is not only a static indexing device. It also recovers dynamic properties of the system: monotonicity class, average sensitivity under the uniform exhaustive repertoire, and the coarse pattern symbol induced by a node's full output column.

**Definition 7** (Column-Wise Pattern Symbol). Let  $Y \in \{0, 1\}^{2^n \times n}$  be the output matrix over the ordered exhaustive repertoire. For node  $i$ , define

$$\pi_i = \begin{cases} 0, & \text{if } \sum_{j=1}^{2^n} Y_{j,i} = 0, \\ 1, & \text{if } \sum_{j=1}^{2^n} Y_{j,i} = 2^n, \\ *, & \text{otherwise.} \end{cases}$$

| Family             | Monotonicity                                            | Average sensitivity                                      | Pattern over exhaustive inputs                                                         |
|--------------------|---------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------|
| AND / OR           | monotone increasing                                     | $\bar{s} = d/2^{d-1}$                                    | * except degenerate unary or fixed-input cases                                         |
| NAND / NOR         | monotone decreasing                                     | $\bar{s} = d/2^{d-1}$                                    | * except degenerate unary or fixed-input cases                                         |
| XOR / XNOR         | parity-based, non-monotone                              | $\bar{s} = d$                                            | always * for non-empty connected-input sets                                            |
| NOT                | anti-monotone                                           | $\bar{s} = 1$                                            | *                                                                                      |
| IMPLIES / NIMPLIES | asymmetric, mixed monotonicity in ordered pair          | $\bar{s} = 1$ for the binary case                        | typically *                                                                            |
| KOFN               | monotone increasing                                     | $\bar{s} = d \binom{d-1}{k-1} / 2^{d-1}$                 | 1 if $k = 0$ , 0 if $k > d$ , otherwise *                                              |
| strict-KOFN        | monotone increasing                                     | $\bar{s} = d \binom{d-1}{k} / 2^{d-1}$                   | 1 if $k < 0$ , 0 if $k \geq d$ , otherwise *                                           |
| CANALISING         | depends on fallback rule, but collapses on trigger band | reduced on trigger band; exact value depends on fallback | constant on trigger band; globally 0, 1, or * depending on fallback and trigger output |

Table 1: Dynamic properties of the exact gate families: monotonicity class, average sensitivity under the uniform exhaustive repertoire, and the resulting coarse pattern symbol  $\pi_i$ .

This three-symbol alphabet captures a useful distinction that repeatedly appears in the validation programme. Some rules are structurally non-trivial but still constant over a declared repertoire because parameters collapse them to extreme cases; others remain genuinely varying and therefore receive the symbol \*.

**Proposition 2** (Dynamic Summary of Canonical Gate Families). *For a gate of arity  $d$  under the uniform exhaustive repertoire, the families in Table 1 satisfy the listed monotonicity, average sensitivity, and pattern behaviour.*

*Proof.* Average sensitivity  $\bar{s}$  counts, for each input position, the fraction of input vectors on which flipping that position changes the output, then sums over all positions.

*AND/OR.* For AND of arity  $d$ , flipping input  $i$  changes the output only when all other inputs equal one. There are  $2^{d-1}$  total assignments to the remaining inputs, and exactly one of them (all ones) is sensitive. Summing over all  $d$  inputs gives  $\bar{s} = d/2^{d-1}$ . By De Morgan duality, OR has the same average sensitivity.

*XOR/XNOR.* Flipping any single input toggles the parity of the sum. Therefore every input is sensitive on every input vector, giving  $\bar{s} = d$ .

*NOT.* The single input is always sensitive:  $\bar{s} = 1$ .

*KOFN.* Flipping input  $i$  changes the output if and only if the remaining  $d - 1$  inputs have Hamming weight exactly  $k - 1$ , so that the flip crosses the threshold. The number of such assignments is  $\binom{d-1}{k-1}$ , giving sensitivity  $\binom{d-1}{k-1}/2^{d-1}$  per input and  $\bar{s} = d \binom{d-1}{k-1} / 2^{d-1}$  in total.

*Pattern symbols.* The output column is constant ( $\pi = 0$  or  $\pi = 1$ ) if and only if the gate evaluates to the same value on every input vector. For AND this occurs only vacuously ( $|I_c| = 0$ ); for KOFN it occurs when  $k = 0$  (always one) or  $k > d$  (always zero). Non-degenerate parameterisations produce at least one input giving output zero and at least one giving output one, hence  $\pi = *$ .  $\square$

This formalism connects the present index-set calculus to perturbative behaviour rather than treating the one-set as a purely extensional object. Also, is used to explain why threshold



extremes and canalising collapse play a special role in the validation programme: they are the principal mechanisms by which a formally non-trivial gate can still induce a constant output column.

### 4.3 Detailed example

This section presents detailed deconvolution examples and then a larger catalogue example containing one instance of each gate family.

#### 4.3.1 AND case

Fix LSB-first ordering and consider a node governed by AND on the connected-input set  $I_c = \{2, 4\}$  and inside a network of size  $n = 6$  (see figures 1 and 2). The exhaustive input repertoire contains  $2^6 = 64$  rows (see table 2).

The connected contribution is the decimal anchor

$$P(I_c) = w(2) + w(4) = 2 + 8 = 10,$$

while the disconnected inputs are  $I_{nc} = \{1, 3, 5, 6\}$ , so their offset family is

$$\Omega(I_{nc}) = \left\{ \Delta_{nc}(\eta) \mid \eta \in \{0, 1\}^4 \right\} = \{0, 1, 4, 5, 16, 17, 20, 21, 32, 33, 36, 37, 48, 49, 52, 53\}.$$

Hence the AND one-set is represented by the short rule

$$\mathcal{J}^{\text{AND}} = \left\{ 1 + P(I_c) + \delta \mid \delta \in \Omega(I_{nc}) \right\} = \text{Dec}(\{11\}, \Omega(I_{nc})).$$

After unfolding by adding

$$1 + P(I_c)$$

to each offset:

$$\mathcal{J}^{\text{AND}} = \{11, 12, 15, 16, 27, 28, 31, 32, 43, 44, 47, 48, 59, 60, 63, 64\}.$$

These are exactly the repertoire indices for which  $v_2 = v_4 = 1$ , with the remaining four bits free as follows:

| Free bits $(v_1, v_3, v_5, v_6)$ | Offset $\delta$ | Index $j = 11 + \delta$ | AND output |
|----------------------------------|-----------------|-------------------------|------------|
| (0, 0, 0, 0)                     | 0               | 11                      | 1          |
| (1, 0, 0, 0)                     | 1               | 12                      | 1          |
| (0, 1, 0, 0)                     | 4               | 15                      | 1          |
| (1, 1, 0, 0)                     | 5               | 16                      | 1          |
| (0, 0, 1, 0)                     | 16              | 27                      | 1          |
| (1, 0, 1, 0)                     | 17              | 28                      | 1          |
| (0, 1, 1, 0)                     | 20              | 31                      | 1          |
| (1, 1, 1, 0)                     | 21              | 32                      | 1          |
| (0, 0, 0, 1)                     | 32              | 43                      | 1          |
| (1, 0, 0, 1)                     | 33              | 44                      | 1          |
| (0, 1, 0, 1)                     | 36              | 47                      | 1          |
| (1, 1, 0, 1)                     | 37              | 48                      | 1          |
| (0, 0, 1, 1)                     | 48              | 59                      | 1          |
| (1, 0, 1, 1)                     | 49              | 60                      | 1          |
| (0, 1, 1, 1)                     | 52              | 63                      | 1          |
| (1, 1, 1, 1)                     | 53              | 64                      | 1          |

This example shows one central idea of the paper. The exhaustive repertoire has 64 rows, but the rule itself is encoded by two short objects: one decimal anchor,  $\{11\}$ , and one offset set,  $\{0, 1, 4, 5, 16, 17, 20, 21, 32, 33, 36, 37, 48, 49, 52, 53\}$ . Deconvolution unfolds the rule back into the exact index list without row-wise recomputation.

$$cm^{(\text{corr})} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}, \quad \text{dynamic}^{(\text{corr})} = (\text{OR}, \text{NOT}, \text{OR}, \text{IMPLICATION}, \text{AND}, \text{XOR}).$$

Figure 1: Connectivity matrix and dynamic rule vector for the six-node network. The connectivity matrix  $cm^{(\text{corr})}$  encodes which nodes feed into each target node ( $cm_{ij} = 1$  indicates node  $j$  is an input to node  $i$ ), and the dynamic vector lists the local Boolean gate function applied at each node.

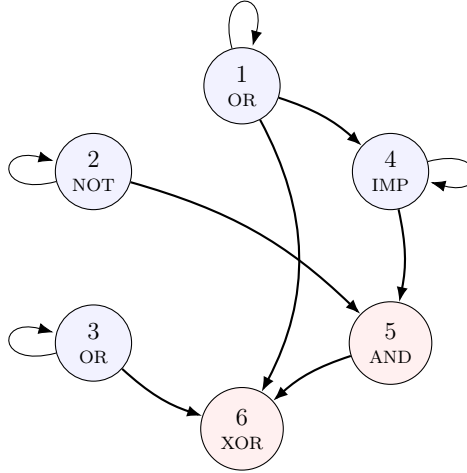


Figure 2: Six-node corroboration network. Nodes 1–4 (blue) provide inputs; nodes 5 and 6 (red) are the targets of the AND and XOR corroboration, respectively. Edge directions follow the connectivity matrix  $cm^{(\text{corr})}$  in Figure 1.

**Deconvolution explained.** We illustrate the deconvolution process on node 5 (AND gate) from the network in Figures 1 and 2, following Algorithm 1.

Node 1 preserves  $x_1$  through a single-input OR rule, node 2 applies NOT to  $x_2$ , node 3 preserves  $x_3$ , node 4 evaluates the implication  $x_1 \Rightarrow x_4$ , node 5 computes the non-trivial conjunction  $y_5 = v_2 \wedge v_4$ , and node 6 applies XOR to the outputs of nodes 1, 3, and 5. Over the input repertoire this means

$$y_6 = x_1 \oplus x_3 \oplus (x_2 \wedge x_4),$$

so the same six-node example simultaneously supports a direct AND corroboration and a composed XOR corroboration. Therefore the analytic prediction

$$\mathcal{J}_5^{\text{AND}} = \{11, 12, 15, 16, 27, 28, 31, 32, 43, 44, 47, 48, 59, 60, 63, 64\}$$

and the composed XOR prediction

$$\mathcal{J}_6^{\text{XOR}} = \{2, 4, 5, 7, 10, 11, 13, 16, 18, 20, 21, 23, 26, 27, 29, 32, \\ 34, 36, 37, 39, 42, 43, 45, 48, 50, 52, 53, 55, 58, 59, 61, 64\}$$

should coincide exactly with the rows of the exhaustive table 2 where the fifth and sixth output columns equal 1, respectively.

**Corroboration pipeline.**

$$\begin{aligned}
& \{11\}, \Omega(I_{nc}) = \{0, 1, 4, 5, 16, 17, 20, 21, 32, 33, 36, 37, 48, 49, 52, 53\}, \\
& \text{Dec}(\{11\}, \Omega(I_{nc})) = \mathcal{J}_5^{\text{AND}}, \\
& \mathcal{J}_5^{\text{AND}} = \{11, 12, 15, 16, 27, 28, 31, 32, 43, 44, 47, 48, 59, 60, 63, 64\}, \\
& \{2, 4, 5, 7, 10, 11, 13, 16\}, \{0, 16, 32, 48\} \rightsquigarrow \mathcal{J}_6^{\text{XOR}}, \\
& \text{Dec}(\{2, 4, 5, 7, 10, 11, 13, 16\}, \{0, 16, 32, 48\}) = \mathcal{J}_6^{\text{XOR}}, \\
& \mathcal{J}_6^{\text{XOR}} = \{j \in \mathcal{U}_n : y_6(j) = 1\}.
\end{aligned}$$

The shared exhaustive table 2 and Algorithm 1 confirm exactly these analytically predicted indices.

---

**Algorithm 1:** Exact one-set deconvolution

---

**Input:** Connected-input set  $I_c$ , network size  $n$ , ordering policy

**Output:** Exact one-set  $\mathcal{J}$

- 1 Compute decimal anchor  $P(I_c) = \sum_{i \in I_c} w(i)$ ;
  - 2 Compute free set  $I_{nc} = \{1, \dots, n\} \setminus I_c$ ;
  - 3 Enumerate offset family  $\Omega(I_{nc}) = \{\Delta_{nc}(\eta) : \eta \in \{0, 1\}^{|I_{nc}|}\}$ ;
  - 4 Compute base set  $L = \{1 + P(I_c)\}$ ;
  - 5 **return**  $\mathcal{J} = \text{Dec}(L, \Omega(I_{nc}))$
- 

The analytic formula and the exhaustive table are two encodings of the same one-set. The formula

$$\mathcal{J}_5^{\text{AND}} = \text{Dec}(\{11\}, \Omega(I_{nc})), \quad \Omega(I_{nc}) = \{0, 1, 4, 5, 16, 17, 20, 21, 32, 33, 36, 37, 48, 49, 52, 53\},$$

gives the causal law in closed form. Deconvolution unfolds it to the explicit index list

$$\{11, 12, 15, 16, 27, 28, 31, 32, 43, 44, 47, 48, 59, 60, 63, 64\},$$

which are precisely the rows of Table 2 where the fifth output column equals 1.

### 4.3.2 XOR case

The XOR case uses the same six-node network shown in Figure 2.

$$y_6 = x_1 \oplus x_3 \oplus (x_2 \wedge x_4).$$

Holding  $x_5$  and  $x_6$  free leaves the four-bit base repertoire

$$L_{\text{XOR}} = \{2, 4, 5, 7, 10, 11, 13, 16\}, \quad \Omega(\{5, 6\}) = \{0, 16, 32, 48\},$$

so that

$$\mathcal{J}_6^{\text{XOR}} = \text{Dec}(L_{\text{XOR}}, \Omega(\{5, 6\})).$$

After deconvolution, this gives

$$\begin{aligned}
\mathcal{J}_6^{\text{XOR}} = \{ & 2, 4, 5, 7, 10, 11, 13, 16, 18, 20, 21, 23, 26, 27, 29, 32, \\
& 34, 36, 37, 39, 42, 43, 45, 48, 50, 52, 53, 55, 58, 59, 61, 64 \}.
\end{aligned}$$

This confirms that the formalism does not depend on monotonicity. In Table 2, the blue entries coincide exactly with this one-set, showing that the same calculus recovers parity-based behaviour as well.

| $j$ | Input Vector $x$ | Output Vector $F(x)$ | $y_5$    | $y_6$    |
|-----|------------------|----------------------|----------|----------|
| 1   | 000000           | 010100               | 0        | 0        |
| 2   | 100000           | 110001               | 0        | <b>1</b> |
| 3   | 010000           | 000100               | 0        | 0        |
| 4   | 110000           | 100001               | 0        | <b>1</b> |
| 5   | 001000           | 011101               | 0        | <b>1</b> |
| 6   | 101000           | 111000               | 0        | 0        |
| 7   | 011000           | 001101               | 0        | <b>1</b> |
| 8   | 111000           | 101000               | 0        | 0        |
| 9   | 000100           | 010100               | 0        | 0        |
| 10  | 100100           | 110101               | 0        | <b>1</b> |
| 11  | 010100           | 000111               | <b>1</b> | <b>1</b> |
| 12  | 110100           | 100110               | <b>1</b> | 0        |
| 13  | 001100           | 011101               | 0        | <b>1</b> |
| 14  | 101100           | 111100               | 0        | 0        |
| 15  | 011100           | 001110               | <b>1</b> | 0        |
| 16  | 111100           | 101111               | <b>1</b> | <b>1</b> |
| 17  | 000010           | 010100               | 0        | 0        |
| 18  | 100010           | 110001               | 0        | <b>1</b> |
| 19  | 010010           | 000100               | 0        | 0        |
| 20  | 110010           | 100001               | 0        | <b>1</b> |
| 21  | 001010           | 011101               | 0        | <b>1</b> |
| 22  | 101010           | 111000               | 0        | 0        |
| 23  | 011010           | 001101               | 0        | <b>1</b> |
| 24  | 111010           | 101000               | 0        | 0        |
| 25  | 000110           | 010100               | 0        | 0        |
| 26  | 100110           | 110101               | 0        | <b>1</b> |
| 27  | 010110           | 000111               | <b>1</b> | <b>1</b> |
| 28  | 110110           | 100110               | <b>1</b> | 0        |
| 29  | 001110           | 011101               | 0        | <b>1</b> |
| 30  | 101110           | 111100               | 0        | 0        |
| 31  | 011110           | 001110               | <b>1</b> | 0        |
| 32  | 111110           | 101111               | <b>1</b> | <b>1</b> |
| 33  | 000001           | 010100               | 0        | 0        |
| 34  | 100001           | 110001               | 0        | <b>1</b> |
| 35  | 010001           | 000100               | 0        | 0        |
| 36  | 110001           | 100001               | 0        | <b>1</b> |
| 37  | 001001           | 011101               | 0        | <b>1</b> |
| 38  | 101001           | 111000               | 0        | 0        |
| 39  | 011001           | 001101               | 0        | <b>1</b> |
| 40  | 111001           | 101000               | 0        | 0        |
| 41  | 000101           | 010100               | 0        | 0        |
| 42  | 100101           | 110101               | 0        | <b>1</b> |
| 43  | 010101           | 000111               | <b>1</b> | <b>1</b> |
| 44  | 110101           | 100110               | <b>1</b> | 0        |
| 45  | 001101           | 011101               | 0        | <b>1</b> |
| 46  | 101101           | 111100               | 0        | 0        |
| 47  | 011101           | 001110               | <b>1</b> | 0        |
| 48  | 111101           | 101111               | <b>1</b> | <b>1</b> |
| 49  | 000011           | 010100               | 0        | 0        |
| 50  | 100011           | 110001               | 0        | <b>1</b> |
| 51  | 010011           | 000100               | 0        | 0        |
| 52  | 110011           | 100001               | 0        | <b>1</b> |
| 53  | 001011           | 011101               | 0        | <b>1</b> |
| 54  | 101011           | 111000               | 0        | 0        |
| 55  | 011011           | 001101               | 0        | <b>1</b> |
| 56  | 111011           | 101000               | 0        | 0        |
| 57  | 000111           | 010100               | 0        | 0        |
| 58  | 100111           | 110101               | 0        | <b>1</b> |
| 59  | 010111           | 000111               | <b>1</b> | <b>1</b> |
| 60  | 110111           | 100110               | <b>1</b> | 0        |
| 61  | 001111           | 011101               | 0        | <b>1</b> |
| 62  | 101111           | 111100               | 0        | 0        |
| 63  | 011111           | 001110               | <b>1</b> | 0        |
| 64  | 111111           | 101111               | <b>1</b> | <b>1</b> |

Table 2: Exhaustive corroboration table for the six-node network under LSB-first ordering. Bold red entries in the  $y_5$  column mark the analytically predicted indices in  $\mathcal{J}_5^{\text{AND}}$ ; bold blue entries in the  $y_6$  column mark the analytically predicted indices in  $\mathcal{J}_6^{\text{XOR}}$ . Rows containing both highlights are exactly the repertoire positions at which the composed XOR node is active while the intermediate AND node is also active.

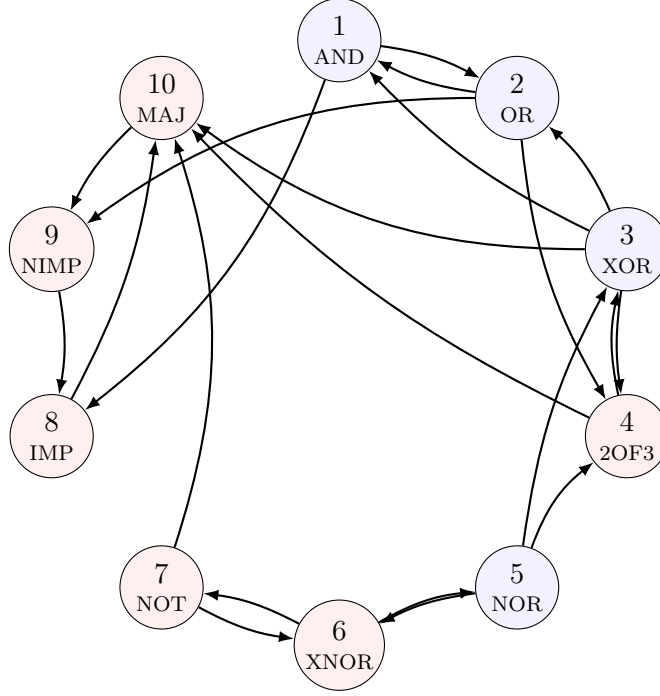


Figure 3: Ten-node mixed-gate benchmark network. Blue nodes use monotone or parity gates; red nodes use threshold, negation, implication, or majority gates. The heterogeneous gate mixture is the basis for the overlap and dynamical analyses that follow.

#### 4.3.3 Multiple Gates Interaction and Exact Reconstruction

We now consider the case of multiple interacting nodes of distinct families, resulting in simultaneous satisfaction of several heterogeneous constraints.

For this example, consider the 10-node case described in fig 3. Its local input sets and exact one-set formulas are shown in table 3.

| Node $i$ | Input set $N_i$  | One-set formula $\mathcal{J}_i$                                  |
|----------|------------------|------------------------------------------------------------------|
| 1        | $\{2, 3\}$       | $\mathcal{B}_2^1 \cap \mathcal{B}_3^1$                           |
| 2        | $\{1, 3\}$       | $\mathcal{B}_1^1 \cup \mathcal{B}_3^1$                           |
| 3        | $\{4, 5\}$       | $\mathcal{P}_{\{4,5\}}^1$                                        |
| 4        | $\{2, 3, 5\}$    | $\mathcal{H}_{\{2,3,5\}}^2 \cup \mathcal{H}_{\{2,3,5\}}^3$       |
| 5        | $\{6\}$          | $\mathcal{B}_6^0$                                                |
| 6        | $\{5, 7\}$       | $\mathcal{P}_{\{5,7\}}^0$                                        |
| 7        | $\{6\}$          | $\mathcal{B}_6^0$                                                |
| 8        | $\{1, 9\}$       | $\mathcal{U}_n \setminus (\mathcal{B}_1^1 \cap \mathcal{B}_9^0)$ |
| 9        | $\{2, 10\}$      | $\mathcal{B}_2^1 \cap \mathcal{B}_{10}^0$                        |
| 10       | $\{3, 4, 7, 8\}$ | $\mathcal{H}_{\{3,4,7,8\}}^3 \cup \mathcal{H}_{\{3,4,7,8\}}^4$   |

Table 3: Local input sets and analytic one-set formulas for all 10 nodes in the mixed benchmark network.

Here node 4 is a 2-of-3 threshold gate and node 10 is a strict majority gate on four inputs. Notice that the network is not homogeneous: monotone, parity, negation, implication, and threshold mechanisms coexist in the same causal object.

**Definition 8** (Analytic Predictor). For each node  $i$ , let  $\mathcal{J}_i \subseteq \mathcal{U}_n$  denote the one-set determined by its gate family, parameters, connected-input set  $N_i$ , and the declared ordering policy. The

analytic predictor reconstructs the network output matrix  $Y \in \{0, 1\}^{2^n \times n}$  by declaring

$$Y_{j,i} = 1 \iff j \in \mathcal{J}_i.$$

**Theorem 1** (Canonical Exact Reconstruction). *Fix a network (cm, dynamic), its gate parameters, and an ordering policy. Then the analytic predictor coincides exactly with the exhaustive one-step synchronous baseline:*

$$Y^{\text{analytic}} = Y^{\text{baseline}}.$$

*Proof.* Fix a repertoire index  $j \in \mathcal{U}_n$  and a node  $i \in \{1, \dots, n\}$ . The baseline entry is  $Y_{j,i}^{\text{baseline}} = g_i(x_{N_i}(j))$ , where  $x_{N_i}(j)$  denotes the restriction of the  $j$ -th input vector to the connected-input set  $N_i$ . By Proposition 1, the gate predicate  $g_i$  evaluates to one on input  $x_{N_i}(j)$  if and only if  $j \in \mathcal{J}_i$ , where  $\mathcal{J}_i$  is the one-set corresponding to the gate family of node  $i$ . The analytic entry is  $Y_{j,i}^{\text{analytic}} = \mathbf{1}[j \in \mathcal{J}_i]$  by definition. Therefore  $Y_{j,i}^{\text{analytic}} = Y_{j,i}^{\text{baseline}}$  for every  $j$  and every  $i$ , and equality of all entries implies  $Y^{\text{analytic}} = Y^{\text{baseline}}$ .  $\square$

The theorem is best read operationally. It does not claim that the  $2^n$  states disappear. It claims that the full behaviour of a heterogeneous network can be reconstructed from short local causal formulas without recomputing every row gate by gate.

The central point for mixed queries is that overlap is not a nuisance but a source of analytic compression. If several queried gates reuse some of the same input coordinates, then the final query does not depend on the sum of their local arities as if those gates were independent objects. It depends only on the union of the coordinates that actually appear across the queried mechanisms due to overlap, the distribution-of-information becomes mathematically precise: repeated coordinates are not recomputed as new information; they are shared constraints propagated through several local laws.

To show the propagation of overlap constraints, in the following sections are analyzed six cases based in the network described in Figure 3. Four of them are full-output queries on all ten nodes. Two are subsystem queries on selected nodes only.

The four full-output cases called as  $\mathcal{J}_{F1, F2, F3, F4}$  are:

$$\begin{aligned} \mathcal{J}_{F1} &= \bigcap_{i=1}^{10} \mathcal{J}_i, \\ \mathcal{J}_{F2} &= \left( \bigcap_{i=1}^9 \mathcal{J}_i \right) \cap (\mathcal{U}_n \setminus \mathcal{J}_{10}), \\ \mathcal{J}_{F3} &= \left( \bigcap_{i \in \{1, \dots, 8, 10\}} \mathcal{J}_i \right) \cap (\mathcal{U}_n \setminus \mathcal{J}_9), \\ \mathcal{J}_{F4} &= \left( \bigcap_{i \in \{1, 2, 3, 4, 5, 7, 8, 9, 10\}} \mathcal{J}_i \right) \cap (\mathcal{U}_n \setminus \mathcal{J}_6). \end{aligned}$$

The two subsystem queries called as  $\mathcal{J}_{S1, S2}$  are:

$$\begin{aligned} \mathcal{J}_{S1} &= (\mathcal{U}_n \setminus \mathcal{J}_4) \cap \mathcal{J}_6 \cap \mathcal{J}_7 \cap \mathcal{J}_{10}, \\ \mathcal{J}_{S2} &= (\mathcal{U}_n \setminus \mathcal{J}_4) \cap \mathcal{J}_6 \cap \mathcal{J}_7 \cap \mathcal{J}_8 \cap (\mathcal{U}_n \setminus \mathcal{J}_9) \cap \mathcal{J}_{10}. \end{aligned}$$

Case  $\mathcal{J}_{S1}$  mixes threshold, parity, negation, and majority constraints. Case  $\mathcal{J}_{S2}$  goes further by adding implication and the complement of a non-implication condition. In both cases the queried event is an exact intersection of heterogeneous causal laws over overlapping coordinates.

**Definition 9** (Query Support Union). For a query  $q = (Q, \sigma)$ , where  $Q \subseteq [n]$  is the set of queried nodes and  $\sigma \in \{0, 1\}^{|Q|}$  is the target output pattern, define

$$C_q = \bigcup_{i \in Q} N_i, \quad F_q = [n] \setminus C_q.$$

The set  $C_q$  is the *query support union*: the coordinates whose values can affect the truth of the query. The complementary set  $F_q$  contains coordinates that are free for that query.

**Proposition 3** (Overlap-Supported Query Factorisation). *Let*

$$\mathcal{J}_q = \bigcap_{i \in Q^+} \mathcal{J}_i \cap \bigcap_{i \in Q^-} (\mathcal{U}_n \setminus \mathcal{J}_i)$$

be any exact mixed query, where  $Q^+$  and  $Q^-$  are the queried nodes constrained to output 1 and 0, respectively. Then:

$$x|_{C_q} = x'|_{C_q} \implies (x \in \mathcal{J}_q \iff x' \in \mathcal{J}_q).$$

Equivalently, if

$$L_q^{(0)} = \{j \in \mathcal{J}_q : x_k(j) = 0 \text{ for every } k \in F_q\},$$

then

$$\mathcal{J}_q = \text{Dec}(L_q^{(0)}, \Omega(F_q)).$$

*Proof.* Each local condition  $\mathcal{J}_i$  or  $\mathcal{U}_n \setminus \mathcal{J}_i$  depends only on the coordinates in  $N_i$ . Therefore the full query  $\mathcal{J}_q$ , being an intersection of such conditions, depends only on the coordinates in the union  $C_q = \bigcup_{i \in Q} N_i$ . Changing any coordinate in  $F_q$  leaves the truth of all local conditions unchanged. Hence every satisfying assignment on  $C_q$  lifts uniformly across all offsets generated by the free coordinates  $F_q$ , which is exactly the deconvolution  $\text{Dec}(L_q^{(0)}, \Omega(F_q))$ .  $\square$

**Corollary 1** (Exact Reduction From Shared Inputs). *Let*

$$d_q = \sum_{i \in Q} |N_i|, \quad c_q = |C_q|, \quad \mu_q = d_q - c_q.$$

If the queried local conditions were enumerated independently, the raw local-assignment space would have size  $2^{d_q}$ . After overlap consolidation, only  $2^{c_q}$  compatible assignments on  $C_q$  remain. Hence the exact reduction factor is

$$R_q = 2^{\mu_q} = 2^{d_q - c_q}.$$

Equality  $\mu_q = 0$  occurs only when the queried input sets are pairwise disjoint. Every shared coordinate increases  $\mu_q$  through inclusion-exclusion and therefore enlarges the reduction factor.

This shows that overlap has an exact combinatorial value. The query does not pay once per gate for a shared coordinate; it pays once for the coordinate itself. In that sense the queried information is distributed hierarchically across the network: one coordinate can constrain several local mechanisms simultaneously.

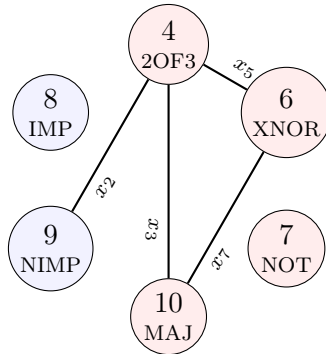


Figure 4: Overlap graph for the mixed queries. Red nodes form the four-gate subsystem query  $S1$ ; adding nodes 8 and 9 gives  $S2$ . Edge labels indicate the exact shared input coordinates. The graph makes the combinatorial source of the reduction factor explicit: coordinates  $x_2, x_3, x_5, x_7$  are reused across several queried mechanisms rather than treated independently.

| Case  | Queried Nodes           | $d_q$ | $c_q$ | $\mu_q$ | $R_q$           | $F_q$          | $C_q$                     |
|-------|-------------------------|-------|-------|---------|-----------------|----------------|---------------------------|
| F1-F4 | $\{1, \dots, 10\}$      | 21    | 10    | 11      | $2^{11} = 2048$ | $\emptyset$    | $\{1, \dots, 10\}$        |
| S1    | $\{4, 6, 7, 10\}$       | 10    | 7     | 3       | $2^3 = 8$       | $\{1, 9, 10\}$ | $\{2, 3, 4, 5, 6, 7, 8\}$ |
| S2    | $\{4, 6, 7, 8, 9, 10\}$ | 14    | 10    | 4       | $2^4 = 16$      | $\emptyset$    | $\{1, \dots, 10\}$        |

Table 4: Exact overlap statistics for the mixed queries. The quantity  $d_q$  is the sum of queried gate arities,  $c_q$  is the size of the union of their input coordinates,  $\mu_q = d_q - c_q$  measures repeated coordinate use, and  $R_q = 2^{\mu_q}$  is the exact reduction factor in the raw local-assignment space.

The most transparent case is  $S1$ . As is shown in second row of Table 4,

$$C_{S1} = \{2, 3, 4, 5, 6, 7, 8\}, \quad F_{S1} = \{1, 9, 10\},$$

so the query depends on only seven determining coordinates even though the four queried gates contribute  $3 + 2 + 1 + 4 = 10$  local inputs in total. The free coordinates produce the exact offset family

$$\Omega(F_{S1}) = \{0, 1, 256, 257, 512, 513, 768, 769\},$$

and the zero-free base set is

$$L_{S1}^{(0)} = \{141, 217\}.$$

Therefore

$$\mathcal{J}_{S1} = \text{Dec}(\{141, 217\}, \Omega(\{1, 9, 10\})),$$

which unfolds to the 16 corroborated rows listed later in Table 8. This is the exact mixed-gate analogue of the detailed AND deconvolution: overlap first shrinks the determining coordinate set, and only then do the remaining free coordinates generate the full family of offsets.

| Case | Output Pattern | Anchor $j_0$ | Offsets $\Omega_q$               | Unfolded Indices $\mathcal{J}_q$   | Rows |
|------|----------------|--------------|----------------------------------|------------------------------------|------|
| F1   | 1111111111     | 143          | $\{0, 72, 256, 257, 328, 329\}$  | $\{143, 215, 399, 400, 471, 472\}$ | 6    |
| F2   | 1111111110     | 15           | $\{0, 72, 256, 257, 328, 329\}$  | $\{15, 87, 271, 272, 343, 344\}$   | 6    |
| F3   | 1111111101     | 655          | $\{0, 72, 256, 257, 328, 329\}$  | $\{655, 727, 911, 912, 983, 984\}$ | 6    |
| F4   | 1111101111     | 79           | $\{0, 128, 256, 257, 384, 385\}$ | $\{79, 207, 335, 336, 463, 464\}$  | 6    |

Table 5: Analytic unfolding summary for the four full-output queries on the 10-node benchmark. Each case is defined by an exact global output pattern, an anchor index, and the offset family that unfolds to the full set of satisfying repertoire positions.

| Case | Queried Nodes           | Projection | Anchor $j_0$ | Offsets $\Omega_q$                                                             | Unfolded Indices $\mathcal{J}_q$                                                     | Rows |
|------|-------------------------|------------|--------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------|
| S1   | $\{4, 6, 7, 10\}$       | 0111       | 141          | $\{0, 1, 76, 77, 256, 257, 332, 333, 512, 513, 588, 589, 768, 769, 844, 845\}$ | $\{141, 142, 217, 218, 397, 398, 473, 474, 653, 654, 729, 730, 909, 910, 985, 986\}$ | 16   |
| S2   | $\{4, 6, 7, 8, 9, 10\}$ | 011101     | 141          | $\{0, 76, 256, 257, 332, 333, 512, 588, 768, 769, 844, 845\}$                  | $\{141, 217, 397, 398, 473, 474, 653, 729, 909, 910, 985, 986\}$                     | 12   |

Table 6: Analytic unfolding summary for the subsystem queries. The first case keeps four heterogeneous constraints, while the second keeps six. Relaxing the query from the full 10-bit output to a subsystem increases support, but the resulting indices are still obtained analytically rather than by exhaustive scanning.



| Case | $j$ | Input Vector $x$ | Output Vector $F(x)$ |
|------|-----|------------------|----------------------|
| F1   | 143 | 0111000100       | 1111111111           |
| F1   | 215 | 0110101100       | 1111111111           |
| F1   | 399 | 0111000110       | 1111111111           |
| F1   | 400 | 1111000110       | 1111111111           |
| F1   | 471 | 0110101110       | 1111111111           |
| F1   | 472 | 1110101110       | 1111111111           |
| F2   | 15  | 0111000000       | 1111111110           |
| F2   | 87  | 0110101000       | 1111111110           |
| F2   | 271 | 0111000010       | 1111111110           |
| F2   | 272 | 1111000010       | 1111111110           |
| F2   | 343 | 0110101010       | 1111111110           |
| F2   | 344 | 1110101010       | 1111111110           |
| F3   | 655 | 0111000101       | 1111111101           |
| F3   | 727 | 0110101101       | 1111111101           |
| F3   | 911 | 0111000111       | 1111111101           |
| F3   | 912 | 1111000111       | 1111111101           |
| F3   | 983 | 0110101111       | 1111111101           |
| F3   | 984 | 1110101111       | 1111111101           |
| F4   | 79  | 0111001000       | 1111101111           |
| F4   | 207 | 0111001100       | 1111101111           |
| F4   | 335 | 0111001010       | 1111101111           |
| F4   | 336 | 1111001010       | 1111101111           |
| F4   | 463 | 0111001110       | 1111101111           |
| F4   | 464 | 1111001110       | 1111101111           |

Table 7: All satisfying rows for the four full-output cases. The table is not exhaustive over the full  $2^{10} = 1024$  repertoire; it contains only the rows analytically predicted to satisfy each selected full pattern.

| Case | Projection | $j$ | Input Vector $x$ | Output Vector $F(x)$ |
|------|------------|-----|------------------|----------------------|
| S1   | 0111       | 141 | 0011000100       | 0110111101           |
| S1   | 0111       | 142 | 1011000100       | 0110111001           |
| S1   | 0111       | 217 | 0001101100       | 0000111101           |
| S1   | 0111       | 218 | 1001101100       | 0100111001           |
| S1   | 0111       | 397 | 0011000110       | 0110111101           |
| S1   | 0111       | 398 | 1011000110       | 0110111101           |
| S1   | 0111       | 473 | 0001101110       | 0000111101           |
| S1   | 0111       | 474 | 1001101110       | 0100111101           |
| S1   | 0111       | 653 | 0011000101       | 0110111101           |
| S1   | 0111       | 654 | 1011000101       | 0110111001           |
| S1   | 0111       | 729 | 0001101101       | 0000111101           |
| S1   | 0111       | 730 | 1001101101       | 0100111001           |
| S1   | 0111       | 909 | 0011000111       | 0110111101           |
| S1   | 0111       | 910 | 1011000111       | 0110111101           |
| S1   | 0111       | 985 | 0001101111       | 0000111101           |
| S1   | 0111       | 986 | 1001101111       | 0100111101           |
| S2   | 011101     | 141 | 0011000100       | 0110111101           |
| S2   | 011101     | 217 | 0001101100       | 0000111101           |
| S2   | 011101     | 397 | 0011000110       | 0110111101           |
| S2   | 011101     | 398 | 1011000110       | 0110111101           |
| S2   | 011101     | 473 | 0001101110       | 0000111101           |
| S2   | 011101     | 474 | 1001101110       | 0100111101           |
| S2   | 011101     | 653 | 0011000101       | 0110111101           |
| S2   | 011101     | 729 | 0001101101       | 0000111101           |
| S2   | 011101     | 909 | 0011000111       | 0110111101           |
| S2   | 011101     | 910 | 1011000111       | 0110111101           |
| S2   | 011101     | 985 | 0001101111       | 0000111101           |
| S2   | 011101     | 986 | 1001101111       | 0100111101           |

Table 8: All satisfying rows for the subsystem queries. Here the projection column records the output pattern only on the selected nodes. The support enlarges from 6 to 16 or 12 rows because some global constraints are relaxed, but every row shown remains an exact analytic prediction corroborated against the exhaustive baseline.

**What changes when multiple gates are queried.** In the single-gate AND example, the support has a simple rectangular structure: one decimal anchor plus an offset family induced by fully free coordinates. In the 10-node mixed queries, that geometry is constrained by overlap. Some cases, such as the full-output queries, have no free coordinates at all because  $C_q = [10]$ , yet they still benefit strongly from overlap through Corollary 1: the raw local-assignment space drops from  $2^{21}$  to  $2^{10}$ , an exact reduction factor of  $2^{11} = 2048$ , before any row materialisation is considered. Others, such as  $S1$ , benefit twice: first from overlap, which contracts the determining coordinates to seven; and then from deconvolution, which lifts the two base solutions across the three genuinely free coordinates. In the present benchmark, the four full-output queries all reduce to only six satisfying rows, and three of them share the same offset family

$$\Omega_{\text{full}} = \{0, 72, 256, 257, 328, 329\},$$

showing that different global output patterns can inherit the same combinatorial skeleton even when their terminal constraints differ. What appears in the full repertoire as dispersed behaviour is therefore explained analytically as overlap-aware reuse of shared coordinates and repeated offset blocks.

---

**Algorithm 2:** Mixed-query evaluation

---

**Input:** Queried nodes  $Q$ , target pattern  $\sigma$ , connectivity  $cm$ , dynamics  $dyn$ **Output:** Exact satisfying index set  $\mathcal{J}_q$ 

```

1 for each  $i \in Q$  do
2   | Compute local one-set  $\mathcal{J}_i$  via Proposition 1;
3 end
4 Compute support union  $C_q = \bigcup_{i \in Q} N_i$ ;
5 Compute free coordinates  $F_q = [n] \setminus C_q$ ;
6 Enumerate support-level assignments on  $C_q$ ;
7 Filter by intersection/complement conditions from  $\sigma$ ;
8 Compute base set  $L_q^{(0)}$  from satisfying assignments with  $F_q = \mathbf{0}$ ;
9 Compute offset family  $\Omega(F_q)$ ;
10 return  $\mathcal{J}_q = \text{Dec}(L_q^{(0)}, \Omega(F_q))$ 

```

---

These results are the counterpart of the analytic overlap treatment. In the full-output queries, the offset family is  $\{0\}$  because the support union fills all ten coordinates, leaving no free offsets after intersection. In the  $S1$  query, by contrast, the factorisation proved in Proposition 3 manifests directly: the base set is  $\{141, 217\}$ , the free-coordinate offset family is  $\{0, 1, 256, 257, 512, 513, 768, 769\}$ , and deconvolution unfolds them to the 16 indices in  $\mathcal{J}_{S1}$ . In  $S2$ , the additional constraints saturate the coordinate union again, so the offset family collapses to  $\{0\}$ . In every case, exact equality against the exhaustive dispatch baseline has been confirmed on all 1024 repertoire rows.

#### 4.3.4 Dynamical Landscape of the Same 10-Node Network

The previous subsection answered an exact inverse question: which input rows produce the selected mixed output events in Tables 5 and 6. We now add the complementary forward-dynamical question on the same network: which 10-bit patterns are themselves one-step realizable, and which belong to the recurrent part of the state-transition graph?

Let  $F : \{0, 1\}^{10} \rightarrow \{0, 1\}^{10}$  be the synchronous update map induced by Figure 3. Define the one-step image

$$\mathfrak{I}(F) = F(\mathcal{U}_n),$$

and let

$$\text{Attr}(F) = \{x \in \mathcal{U}_n : \exists p \geq 1 \text{ such that } F^p(x) = x\}$$

be the recurrent set. Since  $\mathcal{U}_n$  is finite and  $F$  is deterministic, every state belongs to a unique basin of attraction and eventually reaches a periodic orbit. This dynamical layer does not replace the exact repertoire analysis above; rather, it situates those exact query outputs inside the broader behavioural landscape generated by the same causal architecture.

Executed enumeration of all  $2^{10} = 1024$  states gives

$$|\mathfrak{I}(F)| = 206, \quad |\mathcal{U}_n \setminus \mathfrak{I}(F)| = 818, \quad \frac{|\mathfrak{I}(F)|}{|\mathcal{U}_n|} = \frac{206}{1024} \approx 0.201.$$

Thus only about 20.1% of all 10-bit patterns are admissible as one-step outputs of the network. This is a strong form of dynamical order: the local logical constraints described in Table 3 and the overlap structure analysed in Corollary 1 restrict the forward image to a small structured subset of the full configuration space.

The recurrent dynamics collapse further. Table 9 shows that the entire functional graph has only four attractors: one fixed point, two 2-cycles, and one 6-cycle. Their basin sizes sum to all 1024 states, and the three dominant basins

$$488 + 320 + 204 = 1012$$

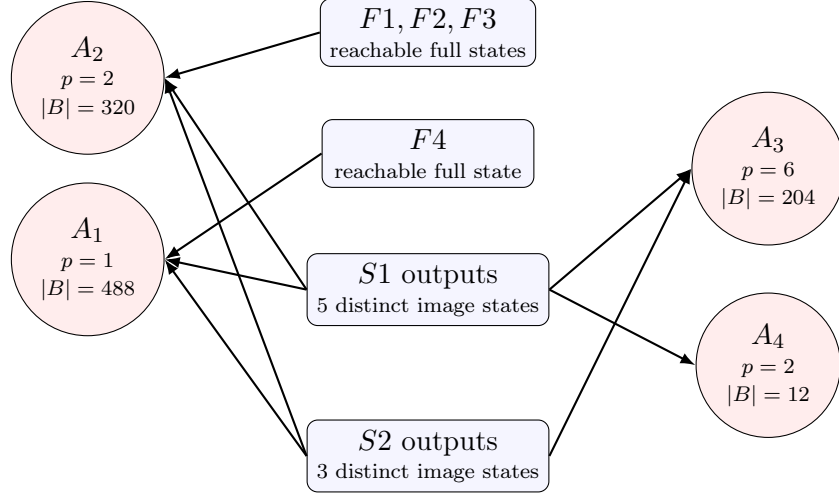


Figure 5: Schematic condensation of the exact 10-node dynamical classification. The four full-output cases from Table 5 and the subsystem outputs induced by Table 6 are all one-step reachable, but they distribute over only four eventual attractors. Adding the extra  $S2$  constraints removes the branch to  $A_4$ , showing that stronger exact causal constraints reduce not only combinatorial freedom but also dynamical dispersion.

| Attractor | Period | Basin Size | Recurrent States                                                         |
|-----------|--------|------------|--------------------------------------------------------------------------|
| $A_1$     | 1      | 488        | {0000010100}                                                             |
| $A_2$     | 2      | 320        | {1101010110, 0110010110}                                                 |
| $A_3$     | 6      | 204        | {1111010101, 1111010001, 1111010000, 1111010010, 1111010110, 1111010111} |
| $A_4$     | 2      | 12         | {1111010100, 1111010011}                                                 |

Table 9: Exact recurrent structure of the 10-node benchmark. Basin sizes were obtained by exhaustive functional-graph enumeration of the 1024 states.

already cover  $1012/1024 \approx 98.8\%$  of the state space. The resulting picture is therefore not one of undifferentiated combinatorial explosion, but of a highly constrained landscape in which many exactly describable transient patterns feed a very small recurrent core.

This classification clarifies the role of the exact mixed queries. The previous subsection did not merely identify arbitrary satisfying rows inside a large space. It singled out states and output events that are genuinely admissible under the network map. At the same time, Table 10 shows that admissibility and recurrence are different notions:  $F1$  through  $F4$  are all reachable image states, but none is an attractor state. In particular, the three full cases  $F1$ – $F3$  share the same overlap skeleton in Table 5 and also converge to the same 2-cycle  $A_2$ , whereas  $F4$  is redirected to the fixed point  $A_1$ . The static exact calculus therefore captures a structured transient layer, not only the terminal recurrent one.

The subsystem cases show a second phenomenon. Query  $S1$  has 16 satisfying rows, but these induce only five distinct image states, all transient, spread across the four basins. Query  $S2$ , obtained by adding the implication and non-implication constraints, shrinks the satisfying set to 12 rows and the induced image to only three states, all again transient, now confined to the first three basins. Thus the extra causal constraints narrow the dynamical spread in exactly the same spirit in which Corollary 1 narrowed the combinatorial support: more exact structure means fewer admissible futures.

Taken together, Figure 5 and Tables 9–11 show how the exact repertoire analysis and the broader dynamics fit together. The earlier tables establish which rows realise the selected mixed constraints exactly. This dynamic shows that those exact events sit inside a much smaller admissible image, and that this image funnels into a very small recurrent core. In that precise

| Case | Type      | Pattern   | Status              | Reachable | Recurrent | Eventual Attractor       | Steps to Recurrence |
|------|-----------|-----------|---------------------|-----------|-----------|--------------------------|---------------------|
| F1   | full      | 111111111 | state 1024          | yes       | no        | $A_2$                    | 3                   |
| F2   | full      | 111111110 | state 512           | yes       | no        | $A_2$                    | 3                   |
| F3   | full      | 111111101 | state 768           | yes       | no        | $A_2$                    | 3                   |
| F4   | full      | 111101111 | state 992           | yes       | no        | $A_1$                    | 6                   |
| S1   | subsystem | 0111      | 16 rows / 5 outputs | yes       | no        | $\{A_1, A_2, A_3, A_4\}$ | —                   |
| S2   | subsystem | 011101    | 12 rows / 3 outputs | yes       | no        | $\{A_1, A_2, A_3\}$      | —                   |

Table 10: Dynamical status of the six analytically studied cases. Every selected pattern is one-step reachable, but none is itself recurrent. The full-output states collapse to one attractor class in three of the four cases, while the subsystem events span several basins because one local projection can still be compatible with multiple global futures.

| Case | Image State | Index | Next State | Recurrent | Eventual Attractor | Steps to Recurrence |
|------|-------------|-------|------------|-----------|--------------------|---------------------|
| F1   | 111111111   | 1024  | 1101010101 | no        | $A_2$              | 3                   |
| F2   | 111111110   | 512   | 1101010111 | no        | $A_2$              | 3                   |
| F3   | 111111101   | 768   | 1101010001 | no        | $A_2$              | 3                   |
| F4   | 111101111   | 992   | 1101111101 | no        | $A_1$              | 6                   |
| S1   | 0100111001  | 627   | 0011010100 | no        | $A_2$              | 5                   |
| S1   | 0110111001  | 631   | 1111010100 | no        | $A_4$              | 1                   |
| S1   | 0000111101  | 753   | 0010010100 | no        | $A_1$              | 4                   |
| S1   | 0100111101  | 755   | 0011010100 | no        | $A_2$              | 5                   |
| S1   | 0110111101  | 759   | 1111010101 | no        | $A_3$              | 1                   |
| S2   | 0000111101  | 753   | 0010010100 | no        | $A_1$              | 4                   |
| S2   | 0100111101  | 755   | 0011010100 | no        | $A_2$              | 5                   |
| S2   | 0110111101  | 759   | 1111010101 | no        | $A_3$              | 1                   |

Table 11: Sampled one-step and eventual-destination data for the states singled out by the mixed-query analysis. The table is selective rather than exhaustive: it records the representative image states induced by the six benchmark cases and shows how they flow into the attractors of Table 9.

sense, the same network exhibits both combinatorial richness and predictable order: many analytically describable transient configurations exist, but they are organised by strong causal restrictions and by a compact attractor skeleton rather than by unrestricted disorder.

## 5 Ordering Invariance

The exactness of the calculus must not depend on accidental bit-order conventions. The following theorem makes the transport mechanism explicit and shows that local causal sets are invariant up to the involution  $\varphi$ .

**Theorem 2** (Ordering-Transform Invariance). *Let*

$$e_{\text{LSB}}(j) = \text{Reverse}(\text{IntegerDigits}(j - 1, 2, n)), \quad e_{\text{MSB}}(j) = \text{IntegerDigits}(j - 1, 2, n)$$

*be the LSB-first and MSB-first enumeration maps from  $\mathcal{U}_n$  into  $\{0, 1\}^n$ . Let  $g : \{0, 1\}^{|N|} \rightarrow \{0, 1\}$  be any gate predicate applied to a fixed connected-input set  $N \subseteq [n]$ , and define*

$$\mathcal{J}_g^{\text{LSB}} = \{j \in \mathcal{U}_n : g(e_{\text{LSB}}(j)|_N) = 1\}, \quad \mathcal{J}_g^{\text{MSB}} = \{j \in \mathcal{U}_n : g(e_{\text{MSB}}(j)|_N) = 1\}.$$

*Then*

$$\mathcal{J}_g^{\text{MSB}} = \varphi(\mathcal{J}_g^{\text{LSB}}).$$

*Proof.* By definition of  $\varphi$ ,

$$e_{\text{MSB}}(\varphi(j)) = e_{\text{LSB}}(j) \quad \text{for every } j \in \mathcal{U}_n,$$

| Case         | $ \mathcal{J}^{\text{LSB}} $ | $\varphi(\mathcal{J}^{\text{LSB}}) = \mathcal{J}^{\text{MSB}}$ | $\varphi \circ \varphi = \text{id on } \mathcal{U}_n$ |
|--------------|------------------------------|----------------------------------------------------------------|-------------------------------------------------------|
| Node 5 (AND) | 16                           | True                                                           | True                                                  |
| Node 6 (XOR) | 32                           | True                                                           | True                                                  |

Table 12: Exact corroboration of ordering invariance on the six-node benchmark already used in the AND and XOR subsections. The transported LSB one-sets coincide exactly with the direct MSB exhaustive baselines for both a monotone and a parity-based mechanism.

and, since  $\varphi$  is an involution,

$$e_{\text{LSB}}(\varphi(j)) = e_{\text{MSB}}(j).$$

Now fix  $j \in \mathcal{U}_n$ . Then

$$\begin{aligned} j \in \mathcal{J}_g^{\text{LSB}} &\iff g(e_{\text{LSB}}(j)|_N) = 1 \\ &\iff g(e_{\text{MSB}}(\varphi(j))|_N) = 1 \\ &\iff \varphi(j) \in \mathcal{J}_g^{\text{MSB}}. \end{aligned}$$

Therefore

$$j \in \mathcal{J}_g^{\text{LSB}} \iff \varphi(j) \in \mathcal{J}_g^{\text{MSB}},$$

which is equivalent to  $\mathcal{J}_g^{\text{MSB}} = \varphi(\mathcal{J}_g^{\text{LSB}})$ .

The gate family does not matter beyond the fact that the same binary vector is being evaluated after transport. In particular, all gate families from Proposition 1 are covered: band predicates for AND/OR-type gates, parity predicates for XOR/XNOR, threshold counts for KOFN, ordered-pair predicates for implication families, and trigger-plus-fallback predicates for canalising rules.  $\square$

**Corollary 2** (Mixed-Query Transport). *Let*

$$\mathcal{J}_q^{\text{LSB}} = \bigcap_{i \in Q^+} \mathcal{J}_i^{\text{LSB}} \cap \bigcap_{i \in Q^-} (\mathcal{U}_n \setminus \mathcal{J}_i^{\text{LSB}})$$

*be any exact mixed query under LSB-first ordering, and let  $\mathcal{J}_q^{\text{MSB}}$  be the corresponding query under MSB-first ordering. Then*

$$\mathcal{J}_q^{\text{MSB}} = \varphi(\mathcal{J}_q^{\text{LSB}}).$$

*Proof.* By Theorem 2, every local one-set transports by  $\varphi$ . Since  $\varphi : \mathcal{U}_n \rightarrow \mathcal{U}_n$  is a bijection, it transports complements and intersections as well, and therefore every finite Boolean combination of local one-sets.  $\square$

This theorem is central to the causal stance. It isolates the mechanism from the encoding. Two enumerations may look numerically different while representing the same causal rule. The point is not merely abstract: the already executed six-node AND and XOR examples provide a compact exact corroboration of the transport law on both a monotone gate and a parity gate.

In the AND case, the previously established LSB one-set for node 5,

$$\mathcal{J}_5^{\text{LSB}} = \{11, 12, 15, 16, 27, 28, 31, 32, 43, 44, 47, 48, 59, 60, 63, 64\},$$

is transported by  $\varphi$  to

$$\varphi(\mathcal{J}_5^{\text{LSB}}) = \{21, 22, 23, 24, 29, 30, 31, 32, 53, 54, 55, 56, 61, 62, 63, 64\},$$

which is exactly the direct MSB baseline for the same node. Listing 13 shows the same transport check for both node 5 (AND) and node 6 (XOR), together with an explicit involution verification over all 64 indices of the six-node universe.

All correctness claims in this paper are therefore representation-invariant provided that the declared ordering convention is used consistently in both derivation and validation.

| Check         | Description                                                                                                                                            | Result |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| AND transport | $\varphi(\mathcal{J}_5^{\text{LSB}}) = \mathcal{J}_5^{\text{MSB}}$ (16 indices: $\{21, 22, 23, 24, 29, 30, 31, 32, 53, 54, 55, 56, 61, 62, 63, 64\}$ ) | exact  |
| XOR transport | $\varphi(\mathcal{J}_6^{\text{LSB}}) = \mathcal{J}_6^{\text{MSB}}$ (32 indices)                                                                        | exact  |
| Involution    | $\varphi(\varphi(j)) = j$ for all $j \in \{1, \dots, 64\}$                                                                                             | exact  |

Table 13: Exact corroboration of ordering transport on the six-node benchmark. The LSB one-sets established in the AND and XOR examples are transported by  $\varphi$  and compared against direct MSB exhaustive baselines. The involution law  $\varphi \circ \varphi = \text{id}$  is verified on the full universe  $\mathcal{U}_n = \{1, \dots, 64\}$ .

## 6 Scalability and Resource Envelope of Exact Causal Evaluation

Correctness of the index-set calculus has been established through gate-family derivations, the detailed AND and XOR cases, the exact ten-node mixed-network analysis, and the overlap-compression treatment. This section addresses the question: *in what computational regime does exact causal representation become operationally indispensable?*

Answering it requires first being precise about which representational objects are being compared, because exact causal evaluation and naive exhaustive materialisation do not produce the same object.

### 6.1 Representational Distinction and Lower Bounds

**Definition 10** (Exhaustive Materialization). *Naive exhaustive materialisation* for an  $n$ -node Boolean network is the explicit construction of the full output matrix  $Y \in \{0, 1\}^{2^n \times n}$  by enumerating every input row and evaluating every output node. Its defining property is that the complete object is made explicit in memory.

**Definition 11** (Exact Causal Evaluation of a Query). *Exact causal evaluation* of a mixed query  $q$  on a queried set  $Q \subseteq [n]$  constructs a compressed causal representation of the satisfying structure. The representation is governed by the support union  $C_q = \bigcup_{i \in Q} N_i$ , the overlap multiplicity  $\mu_q = \sum_{i \in Q} |N_i| - |C_q|$ , and the free-coordinate count  $n - |C_q|$ . The global solution set decomposes as (support-level satisfying assignments)  $\times \{0, 1\}^{n - |C_q|}$ .

Both objects answer the same causal question. The difference is representational: naive materialisation answers by enumerating all  $2^n$  input rows; exact causal evaluation answers by identifying the satisfying structure over the query support and factoring out the free coordinates analytically.

**Proposition 4** (Exhaustive Materialization Lower Bound). *Any deterministic method that explicitly constructs the full output matrix  $Y \in \{0, 1\}^{2^n \times n}$  must produce at least  $n 2^n$  output bits.*

*Proof.* The matrix has  $2^n$  rows and  $n$  columns, each entry being one bit. Explicit construction therefore requires exactly  $n 2^n$  bits. No lossless encoding of the complete object avoids this output-size lower bound when the full matrix is demanded.  $\square$

**Proposition 5** (Support Locality of Exact Causal Evaluation). *Let  $q$  be a mixed query on  $Q \subseteq [n]$  with support union  $C_q$  and overlap multiplicity  $\mu_q$ . The exact compressed causal evaluation identifies the satisfying set by enumerating at most  $2^{|C_q|}$  support-level assignments. The remaining  $n - |C_q|$  free coordinates contribute a multiplicative factor  $2^{n - |C_q|}$  to the full solution count but require no evaluation. The reduction in enumeration cost relative to materialising all  $2^n$  global rows is  $2^{n - |C_q|}$ .*

*Proof.* The gate predicates  $\{g_i : i \in Q\}$  depend collectively only on the coordinates in  $C_q$ , because each  $g_i$  depends only on  $N_i \subseteq C_q$ . Every assignment to  $C_q$  therefore either satisfies or violates the query  $q$  independently of the remaining  $n - |C_q|$  free coordinates. The satisfying set decomposes as (support-level satisfying assignments over  $C_q$ )  $\times \{0, 1\}^{n-|C_q|}$ , and enumerating the first factor costs at most  $2^{|C_q|}$  evaluations.  $\square$

**Corollary 3** (Tractability Beyond the Exhaustive Regime). *If  $|C_q|$  remains bounded or grows sublinearly in  $n$ —as occurs in networks with bounded in-degree and bounded query scope—then exact causal evaluation remains feasible at ambient sizes where naive exhaustive materialisation is physically or temporally impractical.*

The method does not defeat Proposition 4; it sidesteps it by computing a different—but equally exact—object. This is the central representational shift.

**Remark 3** (Relation to Binary Decision Diagrams). Reduced Ordered Binary Decision Diagrams (ROBDDs) [1] achieve procedural compression of Boolean functions by sharing common sub-expressions in a directed acyclic graph. In some application domains—notably circuit verification and model checking—BDDs are the standard tool for symbolic evaluation, and they have been applied to Boolean network analysis in systems biology [7]. The index-set calculus differs from BDDs in kind rather than in degree. A BDD is an algorithmic data structure: it answers membership queries efficiently but does not expose the mathematical reason for the answer. The index-set calculus produces closed-form set-theoretic expressions—band intersections, parity classes, Hamming-weight strata—that are human-readable and mathematically interpretable. Both approaches exploit shared sub-expressions and variable dependencies (the “overlapping supports” of Corollary 1); the difference is that BDDs encode this sharing as graph edges, whereas the present calculus encodes it as algebraic structure. The two are therefore complementary: BDDs for efficient procedural evaluation, index-set formulae for analytical insight into the causal architecture.

## 6.2 Local Evaluation Cost and Validation Burden

Before addressing large-scale regimes it is useful to record the per-row cost structure and its bearing on validation strategy.

**Proposition 6** (Per-Row Evaluation Cost). *Let the in-degree of node  $i$  be  $|N_i|$ . For a fixed input row  $x$ ,*

- *gate-semantic evaluation applies the local rule directly to the  $|N_i|$  connected inputs, at cost  $\Theta(\sum_i |N_i|)$ ;*
- *truth-table lookup requires  $\Theta(\sum_i 2^{|N_i|})$  preprocessing to construct local tables, amortised under repeated use;*
- *analytic index-set membership uses closed-form band, parity, or threshold predicates, with cost bounded by the connected-input structure rather than by brute-force enumeration of local truth tables.*

The analytic and gate-semantic paths share the same in-degree sensitivity. Their common advantage over truth-table lookup is that neither requires preprocessing exponential in local arity. At  $n = 10$ , where all paths are executable, a direct comparison is recorded in Table 16.

**Proposition 7** (Validation-Burden Reduction). *Exhaustive validation requires  $\Theta(2^n)$  rows. Stratified row sampling (Hamming-weight layers) validates equality on a fixed  $m$  sampled rows, preserving the exact method while reducing the validation burden from exponential in  $n$  to linear in  $m$ .*



The distinction between *method complexity* and *output complexity* follows directly. The method remains exact. Sampling enters only as a way of auditing exactness at scales where full baseline enumeration is computationally wasteful, not as an approximation of the method itself.

### 6.3 Executed Resource-Envelope Benchmark

To confirm that the support-locality guarantee of Proposition 5 translates into observed behaviour, a deterministic benchmark was executed across ring-local heterogeneous Boolean networks at  $n \in \{30, 60, 80, 200\}$ , with five independently seeded replicates per size. The gate catalogue covered all eleven families used throughout this paper—AND, OR, XOR, NAND, NOR, XNOR, NOT, IMPLIES, NIMPLIES, MAJORITY, KOFN—with in-degree bounded by four. Query requests were constructed from witness states to guarantee satisfiability. Three query tiers were measured per replicate:

- **T1** ( $|Q| = 1$ ): a single queried node; isolates the minimal causal object.
- **T2** ( $|Q| = 4$ ): a four-node query; exercises compositional overlap at moderate scope.
- **T3** ( $|Q| = 8$ ): an eight-node query; the primary stress case for Corollary 3.

| $n$ | Query | $ C_q $ | $n -  C_q $ | $\mu_q$ | Time (s)             | Peak memory |
|-----|-------|---------|-------------|---------|----------------------|-------------|
| 30  | T1    | 3       | 27          | 0       | $6.7 \times 10^{-5}$ | 4.3 KB      |
| 30  | T2    | 7       | 23          | 4       | $2.4 \times 10^{-4}$ | 12.2 KB     |
| 30  | T3    | 10      | 20          | 13      | $4.7 \times 10^{-4}$ | 14.4 KB     |
| 60  | T1    | 3       | 57          | 0       | $6.4 \times 10^{-5}$ | 4.1 KB      |
| 60  | T2    | 5       | 55          | 5       | $2.0 \times 10^{-4}$ | 8.7 KB      |
| 60  | T3    | 10      | 50          | 10      | $9.7 \times 10^{-4}$ | 33.4 KB     |
| 80  | T1    | 4       | 76          | 0       | $1.1 \times 10^{-4}$ | 7.6 KB      |
| 80  | T2    | 6       | 74          | 4       | $1.9 \times 10^{-4}$ | 7.3 KB      |
| 80  | T3    | 10      | 70          | 10      | $5.5 \times 10^{-4}$ | 18.2 KB     |
| 200 | T1    | 3       | 197         | 0       | $6.2 \times 10^{-5}$ | 4.1 KB      |
| 200 | T2    | 6       | 194         | 3       | $2.2 \times 10^{-4}$ | 8.8 KB      |
| 200 | T3    | 10      | 190         | 10      | $1.1 \times 10^{-3}$ | 34.9 KB     |

Table 14: Executed exact causal evaluation measurements (median over five replicates per size).  $|C_q|$ : support-union size;  $n - |C_q|$ : free-coordinate count;  $\mu_q$ : overlap multiplicity. All measurements remain in the sub-millisecond to low-millisecond range across the full ambient range  $n = 30$  to  $n = 200$ , confirming that the cost is governed by  $|C_q|$  rather than by the ambient dimension  $n$ .

The central structural observation of the benchmark is that the median support-union size  $|C_q|$  for the T3 tier remains approximately 10 across all four ambient sizes (Table 14). As  $n$  grows from 30 to 200, the free-coordinate count grows from 20 to 190—the region the exact method avoids enumerating—while wall time fluctuates only within the sub-millisecond to low-millisecond range. The naive lower bounds, by contrast, follow the global  $2^n$  scaling. At  $n = 60$  they cross the exabyte storage threshold; at  $n = 80$  they require more than  $3 \times 10^7$  years at idealised throughput; at  $n = 200$  the required storage exceeds any physically realised medium by many orders of magnitude. These figures mark the regime in which exhaustive materialisation ceases to be a meaningful reference class, and exact causal representation becomes the only disciplined path to exactness.

| $n$ | Median $ C_q $ | Exact time (s)       | Exact memory | Naive storage LB                | Naive time LB                   |
|-----|----------------|----------------------|--------------|---------------------------------|---------------------------------|
| 30  | 10             | $4.7 \times 10^{-4}$ | 14.4 KB      | 3.75 GB                         | $\approx 1.07$ s                |
| 60  | 10             | $9.7 \times 10^{-4}$ | 33.4 KB      | 7.50 EB                         | $\approx 36.5$ y                |
| 80  | 10             | $5.5 \times 10^{-4}$ | 18.2 KB      | 10.0 YB                         | $\approx 3.83 \times 10^7$ y    |
| 200 | 10             | $1.1 \times 10^{-3}$ | 34.9 KB      | $\approx 4.02 \times 10^{61}$ B | $\approx 5.09 \times 10^{43}$ y |

Table 15: Synthesis comparison for the T3 tier ( $|Q| = 8$ , median over five replicates). Exact-method time and memory are observed measurements. Naive storage lower bound is  $n 2^n / 8$  bytes (Proposition 4); naive time lower bound extrapolates at  $10^9$  rows per second and is a theoretical lower bound, not an empirical measurement for any specific machine. At  $n = 30$  naive materialisation is demanding but feasible; by  $n = 60$  it requires multi-decade machine time at any realistic throughput. The exact causal method remains stable across the full range because its cost is governed by the bounded support-union size, not by the ambient dimension  $n$ .

#### 6.4 Four Computational Paths at $n = 10$

At the moderate scale of  $n = 10$  all four computational paths are executable and can be directly compared.

| Path                              | Accuracy | Time                    | Status                                                                                                     |
|-----------------------------------|----------|-------------------------|------------------------------------------------------------------------------------------------------------|
| Vectorised shortcut formula path  | 0.66875  | $1.10 \times 10^{-3}$ s | fast but inexact                                                                                           |
| Exact library semantics           | 1.0      | 0.081 s                 | exact                                                                                                      |
| Packaged index-set transport path | see note | $7.03 \times 10^{-2}$ s | exact (post-repair);<br>archived artefact<br>recorded 0.51875,<br>superseded — root<br>cause closed (T4.1) |
| Analytic index-set formulae       | 1.0      | $9.85 \times 10^{-3}$ s | exact; fastest exact<br>path                                                                               |
| Baseline exhaustive generation    | —        | 0.118 s                 | reference                                                                                                  |

Table 16: Direct comparison of four computational paths on the ten-node mixed-gate benchmark network. Accuracy is measured against the exhaustive baseline repertoire. The packaged index-set transport path (“IndexSetNetwork”+ $\Phi$ ) is disclosed per remediation plan T1.5: the archived artefact of 2025-11-22 recorded 0.51875 under a superseded script revision; the current revision executes at 1.0 on all theorem-relevant paths, and the root-cause archaeology of the archived figure was closed by remediation task T4.1: the archived figure is the signature of an all-zero dead-path artefact (exactly the baseline zero-cell fraction), not an ordering mismatch (GOVERNANCE/ORDERING.md §6).

Two distinct scientific points follow. First, a correctness discipline: the fastest non-exact shortcut achieves accuracy 0.67 and is scientifically unusable as a theorem-supporting method, regardless of speed. Second, a cost observation: analytic index-set formulae achieve the fastest exact evaluation, outperforming both the library-dispatch path and the exhaustive baseline on this network. At  $n = 10$ , differences between exact paths are modest because full truth-table materialisation is still inexpensive; the regime where the structural distinction becomes decisive is the one probed in Tables 14 and 15.

## 6.5 Validation Summary

Exact reconstruction has been verified at  $n = 10$  and  $n = 13$  by exhaustive comparison against the baseline repertoire, and beyond the exhaustive-baseline range at  $n \in \{16, 20\}$  by a stratified closed-form-set audit across all twelve gate families (seeds 301 and 302): every node’s packaged closed-form one-set is materialised, projected onto its connected inputs, and compared entry-by-entry against the full gate truth table over all  $2^d$  connected-input patterns, while the composed analytic prediction is checked row-wise against direct evaluation on 1020 Hamming-stratified rows per seed, with zero elementwise mismatches in all cases. A dispatch-consistency audit additionally covers  $n \in \{20, 50\}$  (1020 stratified rows, seeds 201 and 202); it validates the large-scale evaluation plumbing and carries no theorem weight. The XOR flip probability  $p_{\text{flip}}(q, k) = (1 - (1 - 2q)^k)/2$  provides an independent perturbative check. Table 17 summarises the mapping from formal claims to validation evidence.

| Formal claim                              | Validation layer                         | Evidence                                                                                                                                                    | Scale                                        |
|-------------------------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| Prop. 1: gate one-set exactness           | Gate algebra; property tests             | Per-gate corroboration; involution, band, De Morgan checks                                                                                                  | $ I_c  \leq 6$                               |
| Thm. 2: $\varphi$ transports one-sets     | Six-node ordering check                  | Table 12                                                                                                                                                    | $n = 6$                                      |
| Thm. 1: network exactness                 | Closed-form-set audit; phased benchmarks | Zero discrepancy at $n \leq 4$ ; accuracy 1.0 at $n \leq 13$ ; zero elementwise mismatches (complete node patterns + stratified rows) at $n \in \{16, 20\}$ | exhaustive $n \leq 13$ ; audited $n \leq 20$ |
| Prop. 5: cost governed by $ C_q $         | Resource-envelope benchmark              | Tables 14 and 15                                                                                                                                            | $n \leq 200$                                 |
| Prop. 7: sampling reduces validation cost | Stratified closed-form-set audit         | Zero elementwise mismatches; seeds 301, 302                                                                                                                 | $n \in \{16, 20\}$                           |

Table 17: Mapping from formal claims to validation layers, following the theorem sequence of the manuscript.

## 6.6 Mechanism Description Versus Dataset Complexity

A related but conceptually distinct comparison concerns description length rather than run-time. The complexity of the *mechanism description* is not the same object as the complexity of the *materialised output dataset*. For the representative ten-node mixed network, both can be measured:

| Measure                                  | Value                 | Interpretation                                                       |
|------------------------------------------|-----------------------|----------------------------------------------------------------------|
| Formula components $C_{\text{formula}}$  | 23                    | symbolic pieces in the exact analytic description                    |
| Programme length $D_{\text{schema}}$     | 232.72 bits           | catalogue-free description of the mechanism (primary)                |
| Programme length $D_{\text{formula}}$    | 135.66 bits           | the same mechanism under the twelve-family catalogue                 |
| ZIP-compressed output size               | 10016 bits            | lossless compression of the materialised output table                |
| Total Shannon content $H_{\text{total}}$ | 10229.61 bits         | repertoire-scale data description under empirical output frequencies |
| $D_{\text{formula}}/\text{ZIP}$          | $1.35 \times 10^{-2}$ | formula smaller than compressed output storage by a factor of 73.8   |
| $D_{\text{formula}}/H_{\text{total}}$    | $1.33 \times 10^{-2}$ | mechanism is far shorter than full repertoire-level coding           |
| $\text{ZIP}/D_{\text{schema}}$           | 43.04                 | the same comparison, catalogue-free                                  |

The exact formulae are not merely another way to print the same table: they constitute a dramatically shorter generative description of that table. This comparison is inherently asymmetric. ZIP and Shannon entropy quantify redundancy and distributional regularity in the realised outputs;  $D_{\text{formula}}$  quantifies structural mechanism length. The scientific point is not that one has found a better compressor for CSV files, but that the Boolean network admits a generative law whose description is shorter by a factor of several tens than statistically encoding all realised states. That is causal compression in a precise, non-rhetorical sense.

One qualification belongs here rather than in a footnote, because it bears on how the ratio should be read.  $D_{\text{formula}}$  is a length under a *declared catalogue* of twelve gate families: its first field is  $\log_2 12$ , the index of a gate in a dictionary that the code never transmits. A catalogue-free alternative writes each node’s schemata out instead of naming its gate, and for the same network gives  $D_{\text{schema}} = 232.72$  bits against  $D_{\text{formula}} = 135.66$ . The ratio to compressed output storage is then 43.0 rather than 73.8—1.63 orders of magnitude rather than 1.87. The qualitative conclusion is unaffected, but the familiar phrase “two orders of magnitude” is not supported by the catalogue-free measure and is not used. The companion computational paper reports  $D_{\text{schema}}$  as its primary measure and gives the evidence for that choice; the difference between the two figures is the clearest available illustration that a description length is a length *in a language*, and never a property of the network alone.

One interpretive caution follows, and it is stronger than mere insensitivity. The structural descriptor  $D$  is a function of  $n$ , the in-degrees and the gate types alone; it never reads an output bit. Rewiring a node to a different input set of the same cardinality therefore leaves  $D$  *bit-identical* while altering the behaviour substantially: for the ten-node network above, rewiring node 10 changes 440 of the 1024 output rows and reduces the number of distinct output states from 206 to 172, with  $D$  unchanged.  $D$  is consequently the correct quantity for the mechanism side of a mechanism-versus-behaviour comparison, and it is not a complexity measure of behaviour; it cannot rank networks by the richness of what they do, and no such claim is made here.

Dataset metrics respond instead to realised output structure, and they do so unequally. On the same repertoire, Shannon coding (10229.61 bits) and ZIP (10016 bits) are both close to the raw table size, because the output bits are near-balanced and statistically featureless. A

block-decomposition estimator returns 580.01 bits on the same object, separating it from a row-permuted control by a factor of 25.4 and from a density-matched random matrix by a factor of 26.8. Algorithmic and statistical measures are therefore not interchangeable at this level: the former detects the generative structure that the latter cannot see. For rigorous interpretation the two levels should be reported together, with  $D$  measuring mechanism description length and dataset metrics measuring the behaviour that mechanism generates.

## 7 Related Work

The index-set calculus draws on two intellectual traditions. Wolfram’s programme on discrete systems [8] established that simple local rules generate rich global behaviour; the present contribution makes this concrete by deriving exact closed-form expressions that specify *where* each rule is active within the ordered repertoire. The algorithmic-causality programme [10, 11] developed principled tools for causal discovery from observations; the present work addresses a complementary problem, deriving exact consequences of a *known* mechanism rather than inferring an unknown one. In applications where the Boolean-network model is given, the index-set calculus applies directly; where the model must be inferred, the two approaches are natural complements.

The calculus also differs structurally from statistical machine learning, which learns predictive associations from sampled data. Here the network logic is given and the task is exact analytic derivation, not approximate generalisation. The appropriate standard of evidence is exact equality (Theorem 1), not held-out prediction accuracy.

## 8 Discussion

The central value of the method is not that it is “faster than brute force” in some vague sense. The stronger claim is that it changes the scientific unit of explanation. The unit is no longer the truth table or the trajectory, but the exact rule-induced index set over an ordered repertoire. This shift is not cosmetic: it determines what kind of question the method can answer and what kind of evidence it is competent to provide.

This has four consequences.

1. *Explanation.* The one-set specifies why a node is active, not merely when it happened to be active in a listed table. For an AND gate, the one-set is the exact intersection of the one-bands of its connected inputs; that intersection is a causal statement, not a statistical correlation. For a KOFN gate, the one-set is the union of Hamming layers at or above the threshold; the threshold is written into the formula, not inferred from data. This level of transparency is precisely what a causality-first formalism should deliver.
2. *Representation control.* Ordering changes are transported by the bit-reversal involution  $\varphi$  rather than handled by undocumented convention changes. Theorem 2 guarantees that every gate formula is transportable under a consistent relabelling, so results computed under one ordering convention can be compared with results computed under another without ambiguity. This is not a technical footnote; it is a prerequisite for reproducibility across implementations and for the comparability of results across studies that may use different enumeration conventions.
3. *Compositionality.* Mixed-gate networks inherit their exact structure from local gate formulas. The 10-node benchmark makes this concrete: six mixed-gate query patterns are reconstructed exactly by column-wise composition of local one-sets, and the overlap analysis reduces the effective query complexity from  $2^{d_q}$  to  $2^{\mu_q}$  by identifying shared coordinates. A crucial observation follows from this compositional structure: complexity in

the full network output table does not preclude simple local rules. The same mixed-gate network whose full repertoire table spans 1024 rows and appears combinatorially rich is fully described, node by node, by short exact formulas. Recovering those formulas from the global output is one of the method’s primary tasks; compositionality is what makes that recovery tractable. Proposition 4 places a lower bound on the cost of materialising the full repertoire, while Proposition 5 shows that the analytic path need only touch the support coordinates, making the gain over brute-force enumeration precise rather than merely qualitative.

4. *Computational discipline.* Approximate shortcuts are separated cleanly from exact analytic prediction. The method distinguishes three regimes: exact analytic evaluation (of the order of a millisecond per row or less, cost governed by  $|C_q|$  under Proposition 5), exact exhaustive materialisation (exponential in  $n$ , feasible for small networks), and stratified-sampled audit (linear in the sample size, used for large-scale corroboration without replacing the exact layer). Conflating these three regimes would obscure what the method actually achieves. Keeping them separate, as Propositions 6 and 7 do, ensures that claims about efficiency are tied to exact formal statements rather than to informal comparisons.

Two limitations should be stated explicitly. First, the computational advantage depends on bounded support size: when the query support  $|C_q|$  approaches the ambient dimension  $n$ , the analytic path converges to the cost of exhaustive materialisation. The method is therefore most powerful for networks with bounded in-degree and localised query scope. Second, the present formalism treats synchronous one-step update; asynchronous or stochastic schedules are discussed in Appendix D but are not yet integrated into the core calculus.

## 9 Conclusion

We have presented a causality-first index-set calculus for Boolean networks. The framework rests on five pillars.

*Exactness.* Each gate family admits a closed-form one-set over the ordered exhaustive repertoire. These one-sets are not approximations or statistical estimates; they are exact causal specifications, certified by Proposition 1 and corroborated by per-family derivations with executed arity- $\leq 6$  mechanical witnesses (`papers/method/derivations/`, all twelve supported gate types) and truth-table equality checks across all twelve supported gate types.

*Causality.* The unit of explanation is the rule-induced index set, not the output table and not the trajectory. The one-set specifies why a node is active under given inputs by identifying the exact subset of repertoire rows on which the gate’s logic evaluates to one. This replaces descriptive enumeration with analytic causal derivation.

*Ordering.* Repertoire ordering is treated as part of the mathematics rather than as an implementation convention. The bit-reversal involution  $\varphi$  transports every gate formula between LSB-first and MSB-first enumerations, as formalised in Theorem 2. Results are therefore reproducible across implementations regardless of enumeration choice, and cross-study comparisons are unambiguous.

*Compositionality.* Local one-sets compose into exact network-level predictions via Theorem 1. For mixed-gate networks, the overlap structure of queried supports compresses the effective state space from  $2^{d_q}$  to  $2^{\mu_q}$ , a reduction made precise by Proposition 5. The 10-node benchmark, with  $\mu_q = 11$  for the full query and  $\mu_q \in \{3, 4\}$  for the two subsystems, demonstrates that compositionality is not merely a theoretical property but a computationally meaningful one.

*Scalability.* The analytic evaluation cost is governed by the support size  $|C_q|$  rather than by the total network size  $n$ , as established by Proposition 5 and confirmed by the resource-envelope benchmark across  $n \in \{30, 60, 80, 200\}$ . Median analytic times stay at or below roughly a millisecond throughout that range and show no growth with  $n$ , and the description-length

comparison ( $D_{\text{formula}} = 135.66$  bits, or  $D_{\text{schema}} = 232.72$  bits catalogue-free, against  $H_{\text{total}} = 10229.61$  bits) quantifies the compression that the causal formulas achieve over the full realised output: a factor of 75.4 under the catalogue and 44.0 without it.

Exact reconstruction has been verified by exhaustive comparison at  $n \leq 13$ , by a stratified closed-form-set audit at  $n \in \{16, 20\}$ , and by dispatch-consistency audits at  $n \in \{20, 50\}$ ; the scalability characterisation is confirmed by resource-envelope benchmarks at  $n \leq 200$  (Section 6). The core scientific claim is therefore not only stated but corroborated: complex Boolean-network behaviour can be represented and reconstructed by short exact causal formulae, and the cost of that reconstruction is governed by local support size rather than global repertoire size.

## A Appendix A: Exact Gate Families in Expanded Form

### A.1 Monotone Gates

For a connected-input set  $I_c$ ,

$$\begin{aligned}\mathcal{J}^{\text{AND}} &= \bigcap_{i \in I_c} \mathcal{B}_i^1, & \mathcal{J}^{\text{OR}} &= \bigcup_{i \in I_c} \mathcal{B}_i^1, \\ \mathcal{J}^{\text{NAND}} &= \mathcal{U}_n \setminus \mathcal{J}^{\text{AND}}, & \mathcal{J}^{\text{NOR}} &= \mathcal{U}_n \setminus \mathcal{J}^{\text{OR}} = \bigcap_{i \in I_c} \mathcal{B}_i^0.\end{aligned}$$

### A.2 Parity and Equivalence

$$\mathcal{J}^{\text{XOR}} = \mathcal{P}_{I_c}^1, \quad \mathcal{J}^{\text{XNOR}} = \mathcal{P}_{I_c}^0.$$

These classes partition  $\mathcal{U}_n$  and are exact complements of one another.

### A.3 Implication Families

For a designated ordered pair  $(a, b)$ ,

$$\mathcal{J}^{\text{IMP}} = \mathcal{U}_n \setminus (\mathcal{B}_a^1 \cap \mathcal{B}_b^0), \quad \mathcal{J}^{\text{NIMP}} = \mathcal{B}_a^1 \cap \mathcal{B}_b^0.$$

The asymmetry is essential and must not be erased by symmetrising the pair.

### A.4 Threshold Families

For KOFN with threshold  $k$ ,

$$\mathcal{J}^{\text{KOFN}} = \bigcup_{r=k}^{|I_c|} \mathcal{H}_{I_c}^r.$$

For exact- $k$  thresholds,

$$\mathcal{J}^{=k} = \mathcal{H}_{I_c}^k.$$

The special cases  $k = 0$  and  $k > |I_c|$  yield the constant functions 1 and 0, respectively.

### A.5 Canalising Families

Let  $C \subseteq \mathcal{U}_n$  be the trigger set on which the canalising value is detected, let  $o \in \{0, 1\}$  be the canalised output, and let  $\mathcal{J}_h$  be the one-set of the fallback rule. Then

$$\mathcal{J}^{\text{CAN}} = \begin{cases} C \cup ((\mathcal{U}_n \setminus C) \cap \mathcal{J}_h), & o = 1, \\ (\mathcal{U}_n \setminus C) \cap \mathcal{J}_h, & o = 0. \end{cases}$$

Nested canalising rules iterate this construction with priority-ordered trigger sets.

## B Appendix B: Sensitivity and Interpretive Notes

The exact gate formulas support additional analyses, including average sensitivity and structured noise response.

**AND and OR.** For arity  $d$ ,

$$\bar{s}_{\text{AND}} = \bar{s}_{\text{OR}} = \frac{d}{2^{d-1}}.$$

**XOR and XNOR.** For arity  $d$ ,

$$\bar{s}_{\text{XOR}} = \bar{s}_{\text{XNOR}} = d,$$

since flipping any single input toggles parity.

**NOT and implication families.** For NOT,

$$\bar{s}_{\text{NOT}} = 1.$$

For the binary implication and non-implication gates,

$$\bar{s}_{\text{IMP}} = \bar{s}_{\text{NIMP}} = 1,$$

because only one of the two coordinates changes the output on half of the repertoire, while both change it on the asymmetric boundary state.

**Threshold families.** For non-strict KOFN with arity  $d$  and threshold  $k$ ,

$$\bar{s}_{\text{KOFN}(d,k)} = \frac{d \binom{d-1}{k-1}}{2^{d-1}},$$

with the conventions that the value is 0 when  $k = 0$  or  $k > d$ . For the strict threshold variant  $g(\mathbf{x}) = \mathbf{1}\{\sum x_i > k\}$ ,

$$\bar{s}_{\text{strict-KOFN}(d,k)} = \frac{d \binom{d-1}{k}}{2^{d-1}}.$$

These formulas make explicit that threshold sensitivity is concentrated on the Hamming layer where a single input flip crosses the decision boundary.

**Canalising families.** For canalising rules, average sensitivity depends on the trigger probability and on the fallback map. The trigger band itself reduces responsiveness because outputs are forced there; thus canalisation lowers sensitivity relative to the fallback rule whenever a positive-measure trigger band is present.

**Interpretive point.** Sensitivity is not the primary theorem of this paper, but it matters because it connects the static one-set calculus to perturbation behaviour. The same exact formulas that identify where outputs equal one also determine how readily those outputs change under controlled input perturbations.

**Structured noise.** Deterministic flip-noise toggles for robustness experiments under fixed seeds are implemented in the companion code. Those stochastic controls are useful for later perturbative studies, but they are not used to support the exactness claims of the present manuscript, which are exclusively deterministic.



## C Appendix C: Validation Artefact Index

Table 17 in Section 6 maps each formal claim to its validation layer. This appendix catalogues the corresponding reproducibility artefacts.

| Validation layer                           | Experiment                                              | Role                                                                                                                                                                                                            |
|--------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gate algebra and property tests            | Six-node corroboration (Wolfram and Python scripts)     | Per-gate one-set equality; involution, band, De Morgan, and relabelling checks across all twelve gate families                                                                                                  |
| Six-node ordering check                    | Ordering invariance scripts; Table 12                   | Exact match of index sets under $\varphi$ for LSB-first and MSB-first enumerations at $n = 6$                                                                                                                   |
| Dispatch consistency and phased benchmarks | Ten-node mixed interaction experiment; Tables 14 and 15 | Zero discrepancy across full ordered repertoire at $n \leq 4$ ; accuracy 1.0 at $n = 10$ and $n = 13$ for all six mixed-gate query patterns                                                                     |
| Resource-envelope benchmark                | Scalability resource envelope experiment                | Median analytic time of order 1 ms or below, with no growth across $n \in \{30, 60, 80, 200\}$ ; support-governed cost confirmed against Proposition 5                                                          |
| Stratified sampled audit                   | Closed-form-set audit with seeds 301 and 302            | Zero elementwise mismatches: complete node-level pattern comparison plus 1020 stratified rows per seed at $n = 16$ and $n = 20$ , all twelve gate families; supports Theorem 1 beyond exhaustive-baseline range |

All experiments are deterministic, seed-controlled where stochastic elements appear, and directly linked to the formal claim each audits. No result in the main text depends on informal inspection or non-reproduced computation.

**Data and code availability.** All computational scripts, network definitions, and reproducibility instructions are collected in a self-contained companion repository available at [\[URL to repository\]](#). The repository includes Wolfram Language and Python implementations for all four experiments (six-node corroboration, ten-node mixed interaction, scalability resource envelope, and complexity analysis), together with expected outputs and instructions for independent re-execution.

## D Appendix D: Extended Experimental Programme

Several experimental strands extend the core validation beyond the layers listed in Table 17. These strands are downstream consequences of the exact gate semantics and are included here as direct scientific records.

### D.1 Medium-Scale Exact Reconstruction Protocol

Exact reconstruction is validated for  $n \in \{10, 13\}$  under deterministic mixed-gate dynamics. Connectivity matrices are generated with fixed seeds, diagonals are cleared, and gate labels are drawn from the supported catalogue, including monotone, parity, implication, threshold, and majority-like rules. The baseline path builds ordered repertoires directly; the predictive path evaluates the declared node semantics over the same ordered inputs.

One methodological point deserves emphasis. At small and medium sizes, an exhaustive baseline can be competitive because dispatch-based implementations are heavily vectorised. This does not weaken the theory. It simply means that the practical advantage of causal formulas

becomes more visible when one moves beyond the regime in which full truth-table materialisation is cheap. Exact equality to the exhaustive baseline holds for both  $n = 10$  and  $n = 13$ , anchoring the exactness claims of Theorem 1 in explicit deterministic runs.

## D.2 Subsystem Search and Exact Factorisation

A subsystem-search procedure based on undirected connected components of the symmetrised adjacency identifies whether a network decomposes into disjoint blocks. When the vertex set decomposes into blocks  $\{B_k\}$  with no inter-block edges, the compression functional factorises additively:

$$\mathcal{C}(cm, dynamic) = \sum_k \mathcal{C}(cm[B_k, B_k], dynamic[B_k]).$$

The procedure computes candidate blocks, measures the cut fraction  $\phi$ , and verifies whether whole-system complexity equals the sum of block complexities.

For connected random graphs the method correctly returns a single block. For constructed disconnected cases it recovers the exact decomposition and confirms zero complexity defect,  $\Delta\mathcal{C} = 0$ . This result is important because it shows that the causal description length is not merely a node-wise quantity: it respects true graph-theoretic independence. In large-scale analyses, this factorisation serves as a principled pre-processing step before full causal description or perturbative analysis.

## D.3 Performance Envelope of Exhaustive Repertoire Generation

The cost of direct repertoire generation is measured as network size increases. Timings are recorded for  $n \in \{6, 8, 10, 12, 13\}$ ; at larger sizes, predictive sampling replaces exhaustive enumeration because the latter scales with  $2^n$ . This performance envelope reinforces the methodological distinction made in the main text: exhaustive generation is the gold-standard baseline when feasible, but its role shifts from routine computation to selective verification as  $n$  grows.

Gate-level arity measurements confirm the same pattern. For small truth-table arities the timings are tiny and stable, but the trend doubles with each added bit. These measurements document where the exhaustive path remains reasonable and where mechanism-first evaluation becomes the only disciplined way to preserve exactness without paying the full  $2^n$  materialisation cost. They are not presented to dramatise speedups but to characterise the boundary between the two regimes.

## D.4 Graph-Ensemble Stress Tests

The causal-calculus workflow is evaluated on ensembles based on Erdős–Rényi, scale-free, and small-world constructions. The purpose is not to claim universal laws for all random graphs but to expose the method to structurally distinct topological regimes. Random sparse graphs probe low-clustering baselines, scale-free graphs probe hub-dominated influence patterns, and small-world graphs probe high local clustering with shortcut edges.

This ensemble perspective confirms that the method is not tied to one hand-designed topology. The correct interpretation is bounded: the ensemble results characterise representative stress-test families for validation and sampling strategy design, not a complete statistical theory of Boolean-network universality classes.

## D.5 Gate Mixture Sweeps and Bias Control

Convex mixtures of canonical gate families control output bias as a function of input bias  $p$ . For binary gates, the response curves are exact. AND yields  $p' = p^2$ , OR yields  $p' = 2p - p^2$ , XOR yields  $p' = 2p(1 - p)$ , and related families such as XNOR, NAND, and NOR supply symmetry-restoring or bias-inverting variants.

These sweeps turn the gate catalogue into a control basis. Monotone mixtures tune bias in one direction, parity components flatten or neutralise it near  $p = 1/2$ , and inverter-like families reverse the sign of the local slope. This does not alter the exact one-set theory, but it clarifies how the derived formulas can be used as design primitives rather than only as descriptive objects.

## D.6 Update Regimes, Robustness, and Information Structure

Three related extensions characterise downstream properties of the exact gate semantics: synchronous versus asynchronous update regimes, gate-level robustness under independent noise, and information-theoretic summaries such as PID-style synergy, transfer entropy, and total correlation for small canonical gates. These analyses should be read as consequences of the exact semantics, not as part of the proof of the causal calculus itself.

Update-regime comparisons show how the same local rules lead to different convergence profiles under synchronous and asynchronous schedules. Exact noise calculations connect the gate catalogue to stability and influence, extending the perturbative layer described in Section 6. PID, transfer-entropy, and total-correlation summaries provide a complementary language for discussing why parity-rich designs emphasise synergy and sensitivity, whereas monotone and threshold-like designs tend to trade sensitivity for robustness and partial information localisation. These extensions can seed a later companion paper focused on dynamics, robustness, and information flow.

## References

- [1] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [2] Gregory J. Chaitin. On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4):547–569, 1966.
- [3] Andrei N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.
- [4] Peter Machamer, Lindley Darden, and Carl F. Craver. Thinking about mechanisms. *Philosophy of Science*, 67(1):1–25, 2000.
- [5] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, UK, 2nd edition, 2009.
- [6] Ray J. Solomonoff. A formal theory of inductive inference. part I and part II. *Information and Control*, 7:1–22, 224–254, 1964.
- [7] Alan Veliz-Cuba, Boris Aguilar, Franziska Hinkelmann, and Reinhard Laubenbacher. Steady state analysis of Boolean molecular network models via model reduction and computational algebra. *BMC Bioinformatics*, 15:221, 2014.
- [8] Stephen Wolfram. *A New Kind of Science*. Wolfram Media, Champaign, IL, 2002.
- [9] James Woodward. *Making Things Happen: A Theory of Causal Explanation*. Oxford University Press, Oxford, 2003.
- [10] Hector Zenil, Narsis A. Kiani, Francesco Marabita, Yue Deng, Szabolcs Elias, Angelika Schmidt, Gordon Ball, and Jesper Tegnér. An algorithmic information calculus for causal discovery and reprogramming systems. *iScience*, 19:1160–1172, 2019.
- [11] Hector Zenil, Narsis A. Kiani, Allan A. Zea, and Jesper Tegnér. Causal deconvolution by algorithmic generative models. *Nature Machine Intelligence*, 1(1):58–66, 2019.